

EDCATALYST

Educational Technology Initiative

University of Petroleum and Energy Studies, Dehradun

EDRecon

Explainable Reconnaissance Framework

Complete Documentation, Command Reference
and Source Code

Dr. Keshav Sinha

School of Computer Science
University of Petroleum and Energy Studies
Dehradun, Uttarakhand, India

Version 1.0.0 . 16 August 2026

Contents

Part I - Manual

- What EDRecon is, and why it exists
- Legal and ethical boundaries
- Installation
- The scope file
- Quick start
- Command reference
- Understanding the output
- Module reference
- The knowledge base
- Reports and evidence
- Classroom use
- Extending EDRecon
- Troubleshooting

Part II - Complete Command Reference

- Setup, knowledge base, passive, scope, system, web
- Output control, reports, full flag list
- Lab-by-lab command sheet
- Maintenance and troubleshooting

Part III - Appendix A: Knowledge Base

- All 40 findings with full explanations

Part IV - Complete Source Code

- All 15 files, annotated

Part I - Manual

EDRecon — User Manual

Explainable Reconnaissance Framework Version 1.0.0

An **EDCatalyst** teaching tool Dr. Keshav Sinha — School of Computer Science, UPES, Dehradun, India
edcatalyst.in

Contents

1. What EDRecon is, and why it exists
2. Legal and ethical boundaries
3. Installation
4. The scope file
5. Quick start
6. Command reference
7. Understanding the output
8. Module reference
9. The knowledge base
10. Reports and evidence
11. Classroom use
12. Extending EDRecon
13. Troubleshooting

1. What EDRecon is, and why it exists

Most reconnaissance tools print observations. nmap tells you port 445 is open. nikto tells you a header is missing. Both are correct, and neither tells a student *why that matters* or *how the tool knows*.

EDRecon is built on a different premise: **the explanation is the product**. Every finding carries six fields:

Field	Question it answers
OBSERVED	What did the tool literally see?
WHAT	What does that mean technically?
WHY	Why does it matter to an attacker or defender?
HOW	What mechanism revealed it?
VERIFY	How do I reproduce this by hand, without the tool?
FIX	What should the defender do?

The VERIFY field matters more than it looks. It always gives a manual command — `dig`, `curl`, `openssl`, `nmap`. A student who runs it learns that EDRecon has no special powers; it automates ordinary protocol conversations. That demystification is the point of the tool.

Design principles

1. **The explanation is decoupled from the code.** All teaching text lives in `knowledge/*.yaml`. Edit it without touching a single line of Python.
2. **Authorisation is structural, not advisory.** A scope file is required for every active module, and there is no override flag.
3. **Confidence is stated, never implied.** A completed TCP handshake is confirmed. A banner string is probable — banners can be edited. Students learn to weigh evidence rather than trust output.
4. **Failures are isolated.** One broken module never ends the run. Errors are collected and reported at the end.
5. **Negative results are reported.** Closed and filtered port counts appear in the output, because "nothing there" is itself a finding.

What EDRecon deliberately does *not* do

It stops at **enumeration and misconfiguration identification**. There is no exploitation, no payload delivery, no credential brute-forcing, no vulnerability chaining. That boundary is intentional: your students already have Metasploit for the exploitation unit, and a recon tool that stays a recon tool is both pedagogically cleaner and not a weapon.

2. Legal and ethical boundaries

Scanning a system you do not own or have written permission to test is a criminal offence in most jurisdictions.

In India, unauthorised access and related conduct fall under the **Information Technology Act, 2000**, principally sections 43 and 66. Penalties include imprisonment. "I was only scanning" and "I was learning" are not defences. Neither is the fact that the target had a weakness.

Note carefully: **passive reconnaissance is not automatically lawful either**. Querying public records is generally fine; aggregating them into a profile of a private individual may not be. EDRecon is designed for infrastructure assessment, not for investigating people.

The rule this tool enforces

Active modules run only against targets listed in a valid, unexpired scope file. This is not a warning banner you can click past — it is a hard gate in the code path. If you find yourself wanting to disable it, that is the signal to stop and get authorisation instead.

Safe practice targets

- Your own lab VMs (Metasploitable 2/3, DVWA, OWASP Juice Shop, VulnHub)
- Systems you personally own
- Deliberately public scanning targets such as `scanme.nmap.org`, within the terms its operator publishes
- Client or employer systems **with written authorisation**

3. Installation

Requirements

- Python 3.8 or newer
- Linux, macOS, or Windows (developed and tested on Kali Linux)

Steps

```
# 1. Place the edrecon folder wherever you keep tools
cd ~/tools/edrecon

# 2. Create a virtual environment (as your normal user - NOT with sudo)
python3 -m venv .venv

# 3. Activate it
source .venv/bin/activate          # Linux / macOS
# .venv\Scripts\activate          # Windows

# 4. Install dependencies (no sudo - the venv is yours)
pip install -r requirements.txt

# 5. Confirm it runs
python3 edrecon.py --version
```

Never use `sudo pip` inside an activated virtual environment. `sudo` runs as root with root's environment, bypasses the venv entirely, and writes into system directories that your distribution's package manager owns. On Kali and Debian this produces the `uninstall-no-record-file` error and leaves packages scattered across two Python installations. If you hit permission errors, the venv directory is root-owned — delete it, recreate it without `sudo`, and reinstall.

Dependencies

Package	Required?	Used for
PyYAML	Yes	Scope file and knowledge base parsing
requests	Yes	HTTP modules, certificate transparency
dnspython	Yes	DNS reconnaissance
beautifulsoup4	Recommended	Form analysis
python-whois	Recommended	WHOIS lookups

Missing optional packages degrade gracefully — that module reports an error and the scan continues.

Verifying the install

```
python3 edrecon.py --concepts    # should list ~85 concept tags
python3 edrecon.py --kb         # should print the full knowledge base
```

Both work with no target and no network connection.

4. The scope file

The scope file is the authorisation record. Copy `scope.example.yaml` and edit it.

```
engagement:
  name: "Ethical Hacking Lab - Unit 2 Reconnaissance"
  authorised_by: "Dr. Keshav Sinha, School of Computer Science, UPES"
  contact: "keshav.sinha@ddn.upes.ac.in"
  expires: "2026-12-31"
  notes: >
    Isolated lab environment. Metasploitable 3 on a host-only network.

in_scope:
  hosts:
    - "192.168.56.101"
    - "192.168.56.102"
  networks:
    - "192.168.56.0/24"
  domains: []

out_of_scope:
  - "192.168.56.1"          # VirtualBox host adapter
  - "192.168.56.100"       # your Kali VM
```

Rules

- All four engagement fields are **required**. EDRecon refuses to start without them.
- `expires` must be YYYY-MM-DD. **An expired scope is refused**, exactly as real authorisation lapses. This is the field students most often overlook, and that lesson is deliberate.
- At least one of `hosts`, `networks`, or `domains` must be non-empty.
- `domains` automatically includes subdomains.
- `out_of_scope` **always wins**. A host inside an in-scope network is still refused if it is explicitly excluded. This models the real situation where one device on an in-scope subnet must not be touched.

Testing your scope file

```
python3 edrecon.py -t 192.168.56.101 --system --scope scope.yaml
```

The authorisation block prints before anything is scanned. Read it. If it does not say what you expect, fix the file before continuing.

5. Quick start

Passive only — no scope file needed

```
python3 edrecon.py -t example.com --passive-only
```

No packets reach the target. Safe to run against any domain.

Full scan of an authorised lab target

```
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --all
```

System scan with a custom port range

```
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
  --system --ports 1-1024
```

Web scan with content discovery

```
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
  --web --discover
```

Browse the knowledge base — no target at all

```
python3 edrecon.py --kb
python3 edrecon.py --kb-search clickjacking
python3 edrecon.py --concepts
```

6. Command reference

Target selection

Flag	Meaning
-t, --target	Domain, hostname, or IP address
-s, --scope	Path to the authorisation scope file
-u, --url	Explicit URL for the web phase

Phases

Flag	Meaning
--passive-only	Passive reconnaissance only. No scope file needed.
--system	Active system scan (host discovery, ports, TLS)
--web	Active web scan (headers, cookies, forms)
--all	All three phases

With no phase flag, EDRecon runs --passive-only. Safe by default.

Scan options

Flag	Default	Meaning
--ports	top	top, 1-1024, or 22,80,443
--threads	50	Concurrent scan threads
--timeout	3.0	Per-connection timeout (seconds)

Flag	Default	Meaning
--delay	0.35	Delay between web requests
--no-banners	off	Skip banner grabbing
--axfr	off	Attempt zone transfer (active, needs scope)
--discover	off	Run content discovery in the web phase
--wordlist	built-in	Custom wordlist file
--max-paths	400	Cap on paths probed
--no-whois	off	Skip WHOIS
--no-ct	off	Skip certificate transparency

Output

Flag	Meaning
--brief	One line per finding (suppresses the teaching layer)
--no-explain	Observations without WHAT/WHY/HOW
--outdir	Report directory (default reports)
--no-color	Disable ANSI colour
--no-report	Skip file output

Knowledge base

Flag	Meaning
--kb	Print the entire knowledge base
--kb-search TERM	Search the knowledge base
--concepts	List all concept tags
--verify-log PATH	Verify an evidence log's hash chain

7. Understanding the output

Anatomy of a finding

```
[WEB-007] Version control directory exposed ..... high
-----
SOURCE  ACTIVE module 'content_discovery' | confidence: confirmed
OBSERVED http://target/.git/HEAD -> HTTP 200, 23 bytes.
WHAT    An exposed repository often allows reconstruction of the
        entire application source, including commit history.
WHY     History is the real prize: credentials and keys that were
        committed and later removed remain recoverable.
HOW     Request to the version control path returned an existing
        resource.
VERIFY  curl -sI http://<target>/.git/HEAD
FIX     Deny access to dot-directories at the web server. Rotate any
        credential that was ever committed.
REFS    CWE-527 | CWE-540
TAGS    source-disclosure . version-control . secret-leakage
```

Severity

Level	Meaning
info	Neutral fact. Not a weakness by itself.
low	Minor hygiene issue. Useful only in combination.
medium	Real weakness. Meaningfully expands attack surface.
high	Serious weakness. Direct path toward compromise.
critical	Compromise likely trivial from here.

Confidence — read this carefully

This is the field that separates a practitioner from someone reading tool output credulously.

Level	Meaning
confirmed	Directly proven. The target told us, or a full protocol exchange completed.
probable	Strongly indicated by a distinctive signature that is rarely wrong.
inferred	A best guess from indirect signals. Defeatable by misconfiguration or deliberate deception.

An open port is confirmed — the handshake completed. A version number from a banner is probable — administrators can and do edit banners. Never report an inferred finding as fact.

The "WHAT THIS SCAN TAUGHT YOU" block

Every finding carries concept tags. At the end of a run, EDRecon lists every concept the scan touched. Use it as a revision checklist: any tag you cannot explain is a gap, and `--kb-search <tag>` will fill it.

8. Module reference

Passive modules

Passive means **no packet reaches the target**. Queries go to resolvers, registries and public logs. The target's own logs stay empty.

Module	Findings	What it does
dns_recon	DNS-001–DNS-009	A/AAAA, NS, MX, TXT, SPF, DMARC
whois_lookup	WHO-001, WHO-002	Registrar, dates, contacts, expiry
cert_transparency	DNS-010	Subdomains from public CT logs

Certificate transparency deserves special attention in teaching. Every publicly trusted certificate issued since 2018 is written to a public, append-only log. Querying crt.sh enumerates subdomains without sending a single packet to the target — the organisation published that data itself, as an unavoidable side effect of obtaining certificates.

Active system modules

Module	Findings	What it does
host_discovery	HOST-001	TCP-based liveness probe
port_scan	PORT-001–PORT-006	TCP connect scan, banner grabbing
tls_audit	TLS-001–TLS-009	Certificates, versions, ciphers

On the scan type: this is a TCP *connect* scan — the full three-way handshake completes before the socket closes. That makes it reliable and unprivileged, but also loud: the connection appears in the target's application logs. A SYN scan avoids that by never completing the handshake, but needs raw sockets and therefore root. The stealth-versus-privilege trade-off is worth demonstrating directly in class.

On AXFR: zone transfer is the one DNS check labelled **active**, because the request goes straight to the target's authoritative nameserver. It needs a scope file.

Active web modules

Module	Findings	What it does
http_audit	HTTP-001–HTTP-003, HDR-001–HDR-006, WEB-001, WEB-004–WEB-006	Fingerprinting, headers, cookies, forms
content_discovery	WEB-002, WEB-003, WEB-007	Wordlist path discovery

On the baseline in content discovery: a naive scanner reports every HTTP 200 as a hit, which produces pages of noise against any server with a custom 404 page. Before probing anything, EDRecon requests two paths that certainly do not exist and records the response — status, length, body hash. Every subsequent response is compared against that baseline. When soft-404 behaviour is detected, EDRecon says so explicitly in the output.

Content discovery is rate-limited by construction. Unthrottled forced browsing is indistinguishable from a denial-of-service attempt.

9. The knowledge base

All explanatory text lives in knowledge/:

```
knowledge/
■■■ dns.yaml      DNS, SPF, DMARC, WHOIS
■■■ ports.yaml    Hosts, ports, services
■■■ tls.yaml      Certificates, protocol versions, ciphers
■■■ web.yaml      HTTP, headers, cookies, content discovery
```

Editing an explanation

```
HDR-001:
title: "Strict-Transport-Security header missing"
what: >
  The server does not instruct browsers to use HTTPS exclusively.
why_it_matters: >
  Without HSTS, a user typing the bare hostname makes an initial plain
  HTTP request that an attacker on the path can intercept.
how_detected: "Response headers checked for Strict-Transport-Security."
how_to_verify: "curl -sI https://<target>/ | grep -i strict-transport"
how_to_fix: >
  Send: Strict-Transport-Security: max-age=31536000; includeSubDomains
severity: medium
references: ["RFC 6797", "CWE-319"]
concept_tags: ["hsts", "ssl-stripping", "transport-security"]
```

Save the file and rerun. No code changes, no restart beyond the next invocation.

Why this separation matters

- **Translate it.** Produce a Hindi knowledge base without forking the code.
- **Version the pedagogy separately.** Explanations improve on a different cycle from scanner logic.
- **Set it as reading.** Students can browse `--kb` before ever running a scan.
- **Let students write entries.** Adding a well-researched KB entry is an excellent assessed exercise — and a considerably better test of understanding than running a scan.

10. Reports and evidence

Every scan produces three artefacts in reports/:

File	Purpose
report_<target>_<timestamp>.html	Styled report with the full teaching layer
report_<target>_<timestamp>.json	Machine-readable findings
evidence_<target>_<timestamp>.json l	Append-only, hash-chained event log

The evidence log

Each entry contains the SHA-256 hash of the previous entry. Altering any earlier line invalidates every line after it — the same principle as a forensic hash chain, at a scale students can inspect by hand.

```
python3 edrecon.py --verify-log reports/evidence_target_20260816.jsonl
```

```
[+] Evidence chain intact - no entry has been altered.
```

If a line is modified:

```
[x] Chain BROKEN at line 2: content hash mismatch
```

Classroom exercise: have students run a scan, open the JSONL in an editor, change one recorded value, and re-verify. Watching the chain break teaches integrity verification far better than a lecture slide about it. This also bridges directly into your Digital Forensics course.

11. Classroom use

Suggested lab progression

Lab 1 — Passive reconnaissance (no scope file) Run `--passive-only` against the institution's own domain. Discuss what was learned without touching a single target system. Have students verify three findings by hand using the `VERIFY` commands.

Lab 2 — Writing a scope file Before any active scanning, each student writes and defends a scope file. Deliberately give them an expired one first and let the tool refuse. The lesson lands better from a refusal than from a slide.

Lab 3 — Active system scanning Scan Metasploitable 3. Compare EDRecon's output against `nmap -sv`. Where do they agree? Where does confidence differ, and why?

Lab 4 — Web scanning and content discovery Run `--web --discover`. Discuss the baseline mechanism: why is comparing against a known-bad path necessary?

Lab 5 — Evidence integrity Verify a log, tamper with it, verify again. Connect to chain of custody.

Lab 6 — Extend the tool Students add a knowledge base entry or a module. Assessed on the quality of the explanation, not the code.

Assessment ideas

- Give students a JSON report and ask them to write the executive summary.
- Give a finding with confidence: `inferred` and ask what evidence would raise it to `confirmed`.
- Ask students to identify which findings in a report would change if the target sat behind a CDN.
- Have students critique a KB entry's `why_it_matters` field and improve it.

Demonstrating the safety model

Run this in front of the class:

```
python3 edrecon.py -t scanme.nmap.org --system --scope scope.yaml
```

```
[x] TARGET NOT AUTHORISED: 'scanme.nmap.org' is not covered by the  
scope file 'scope.yaml'.
```

Then show there is no `--force` flag in `--help`. The point is that professional tooling should make the wrong thing hard, not merely discouraged.

12. Extending EDRecon

Adding a knowledge base entry

Add to any knowledge/*.yaml:

```
SNMP-001:
title: "SNMP responds to default community string"
what: >
  The service accepted 'public' as a community string.
why_it_matters: >
  Community strings are effectively passwords sent in cleartext in
  SNMPv1 and v2c. A default value exposes the full device
  configuration, interface list and routing table.
how_detected: "An SNMP GET for sysDescr.0 using the string 'public'."
how_to_verify: "snmpwalk -v2c -c public <target> system"
how_to_fix: >
  Change the community string, restrict by source IP, migrate to
  SNMPv3 with authentication and privacy.
severity: high
references: ["CWE-1188", "RFC 3411"]
concept_tags: ["snmp", "default-credentials", "cleartext-protocol"]
```

Adding a module

Create modules/system/snmp_check.py:

```
from core.finding import Finding
from core.knowledge import kb

MODULE = "snmp_check"

def _mk(fid, target, observation, evidence=None, **kw):
    f = Finding(id=fid, module=MODULE, target=target, title="",
                observation=observation, evidence=evidence or {},
                passive=False, **kw)
    return f.enrich(kb().get(fid))

def check(host, scope, timeout=3, evidence_log=None):
    scope.assert_in_scope(host)          # MANDATORY - first line, always
    findings = []
    # ... your logic ...
    return findings
```

Then wire it into the relevant phase function in `edrecon.py` using `run_module()`, which handles error isolation for you.

The three rules for a new active module

1. `scope.assert_in_scope(host)` is the **first executable line**.
2. Return `Finding` objects. Never `print()` directly.
3. Raise `RuntimeError` with a clear message on failure — `run_module()` catches it, records it, and lets the scan continue.

13. Troubleshooting

ModuleNotFoundError **after installing**

The virtual environment is almost certainly not activated. Check your prompt — it should show (.venv). If not:

```
source .venv/bin/activate
```

This must be done in every new terminal session.

Permission denied **when creating the venv**

The directory is root-owned, usually because an installer ran under sudo.

```
sudo chown -R $USER:$USER .  
rm -rf .venv  
python3 -m venv .venv      # no sudo
```

error: uninstall-no-record-file

You ran `sudo pip install` on a Debian-family system. `sudo` bypassed the venv and pip tried to remove a package that apt owns. Recreate the venv and install without `sudo`.

AUTHORISATION EXPIRED

Working as designed. Update `engagement.expires` in the scope file — but only if you actually still hold authorisation.

TARGET NOT AUTHORISED

The target is not in the scope file, or is explicitly excluded. Add it only if you hold written permission. There is no override, and that is deliberate.

crt.sh returns HTTP 429

Rate limited. Wait a minute and retry, or use `--no-ct`.

Content discovery finds nothing on a known-vulnerable target

Check whether the output mentions soft-404 behaviour. If the server returns 200 for everything with identical bodies, all responses are filtered as baseline matches. Try a different wordlist, or inspect responses manually with `curl -I`.

The scan is very slow

Reduce `--timeout`, raise `--threads`, or narrow `--ports`. On a lab network `--timeout 1 --threads 100` is usually comfortable. Do not raise thread counts against systems you do not control.

TLS audit fails on a self-signed lab certificate

It should not — certificate verification is disabled deliberately, because the point is to *observe* the certificate, not to trust it. If the handshake itself fails, the service may not speak TLS on that port.

Appendix — Finding ID index

Range	Domain
DNS-001-DNS-010	DNS records, SPF, DMARC, subdomains
WHO-001-WHO-002	WHOIS registration
HOST-001	Host liveness
PORT-001-PORT-006	Ports and services
TLS-001-TLS-009	Certificates and TLS configuration
HTTP-001-HTTP-003	HTTP fingerprinting
HDR-001-HDR-006	Security headers and cookies
WEB-001-WEB-007	Web surface and content discovery

Full detail for any ID:

```
python3 edrecon.py --kb-search DNS-008
```

EDRecon — an EDCatalyst teaching tool. Dr. Keshav Sinha, UPES, Dehradun, India. Authorised testing only.

Part II - Complete Command Reference

EDRecon — Complete Command Reference

EDCatalyst · Dr. Keshav Sinha · UPES, Dehradun, India

Every command assumes you are in the edrecon directory with the virtual environment active:

```
cd ~/edrecon && source .venv/bin/activate
```

1. Setup and verification

Command	What it does
<code>python3 -m venv .venv</code>	Create the virtual environment (never with sudo)
<code>source .venv/bin/activate</code>	Activate it — needed in every new terminal
<code>deactivate</code>	Leave the virtual environment
<code>pip install -r requirements.txt</code>	Install dependencies (no sudo)
<code>python3 edrecon.py --version</code>	Print version and attribution
<code>python3 edrecon.py --help</code>	Full flag list
<code>python3 edrecon.py</code>	Print help (no target given)

Offline self-test — no network, no target:

```
python3 edrecon.py --concepts    # ~85 concept tags should list
python3 edrecon.py --kb         # full knowledge base prints
```

2. Knowledge base — no target required

	What it does	
<code>-kb</code>	Print every knowledge base entry	
<code>-kb \</code>	less	Page through it
<code>-concepts</code>	List all concept tags	
<code>-kb-search</code>	Search titles and body text	

Search examples:

```
python3 edrecon.py --kb-search clickjacking
python3 edrecon.py --kb-search zone-transfer
python3 edrecon.py --kb-search forward-secrecy
python3 edrecon.py --kb-search cleartext
python3 edrecon.py --kb-search DNS-008          # search by finding ID
python3 edrecon.py --kb-search TLS              # all TLS entries
python3 edrecon.py --kb-search spf
python3 edrecon.py --kb-search csrf
```

Export the knowledge base as a handout:

```
python3 edrecon.py --kb --no-color > edrecon_knowledge_base.txt
```

3. Passive reconnaissance — no scope file needed

No packets reach the target. Safe against any domain.

Command	What it does
python3 edrecon.py -t DOMAIN --passive-only	Full passive sweep
python3 edrecon.py -t DOMAIN	Same — passive is the default

Variations:

```
# Full passive scan
python3 edrecon.py -t upes.ac.in --passive-only

# Skip certificate transparency (faster, or if crt.sh is rate limiting)
python3 edrecon.py -t upes.ac.in --passive-only --no-ct

# Skip WHOIS
python3 edrecon.py -t upes.ac.in --passive-only --no-whois

# DNS only — fastest
python3 edrecon.py -t upes.ac.in --passive-only --no-ct --no-whois

# One line per finding
python3 edrecon.py -t upes.ac.in --passive-only --brief

# Longer DNS timeout on a slow link
python3 edrecon.py -t upes.ac.in --passive-only --timeout 10
```

4. Scope file management

The scope file is required for every active module. There is no override.

```
# Create your own from the template
cp scope.example.yaml scope.yaml
nano scope.yaml

# Find your Metasploitable IP first (run on the target VM)
ip a

# Confirm the target is reachable before scanning
ping -c 2 192.168.56.101
```

Test that the guard actually refuses — run these in class:

```
# 1. No scope file at all
python3 edrecon.py -t 192.168.56.101 --system

# 2. Target not listed in the scope file
python3 edrecon.py -t scanme.nmap.org --system --scope scope.yaml

# 3. Expired authorisation
sed 's/expires: ./expires: "2020-01-01"/' scope.yaml > /tmp/dead.yaml
python3 edrecon.py -t 192.168.56.101 --system --scope /tmp/dead.yaml

# 4. Prove there is no bypass flag
python3 edrecon.py --help | grep -i force      # returns nothing
```

Inspect scope logic directly:

```
python3 -c "
import sys; sys.path.insert(0, '.')
from core.scope_guard import ScopeGuard
s = ScopeGuard('scope.yaml')
print(s.summary())
for t in ['192.168.56.101', '192.168.56.1', '8.8.8.8']:
    print(f'{t:20} -> {"ALLOWED" if s.is_in_scope(t) else "REFUSED"}')
"
```

5. Active system scanning

Requires `--scope`. Packets reach the target and are logged by it.

Command	What it does
<code>--system</code>	Host discovery + port scan + TLS audit

Port specification:

```
# Default: ~48 curated well-known ports
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --system

# Explicit list
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
    --system --ports 21,22,23,80,443,3306

# Range
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
    --system --ports 1-1024

# Full sweep (slow – tune timeout and threads)
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
    --system --ports 1-65535 --threads 200 --timeout 1

# Mixed
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
    --system --ports 1-1024,3306,8080,8443
```

Speed tuning (lab network):

```
# Fast
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
    --system --ports 1-10000 --threads 200 --timeout 0.5

# Slow and reliable (across a WAN or through a firewall)
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
    --system --threads 20 --timeout 8
```

Other system options:

```
# Skip banner grabbing (quieter, faster)
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
    --system --no-banners

# Zone transfer attempt (ACTIVE – needs scope)
python3 edrecon.py -t lab.example.local --scope scope.yaml --axfr
```

6. Active web scanning

```
# Auto-detect web ports from the system scan
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --web

# Explicit URL
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
    --web -u http://192.168.56.101:8080/

# HTTPS
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
    --web -u https://192.168.56.101/

# Specific application path
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
    --web -u http://192.168.56.101/dvwa/
```

Content discovery:

```
# Built-in wordlist (~70 paths)
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --web --discover

# Bundled wordlist file
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
  --web --discover --wordlist wordlists/common.txt

# SecLists, capped
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
  --web --discover \
  --wordlist /usr/share/wordlists/dirb/common.txt \
  --max-paths 1000

# Fast on an isolated lab
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
  --web --discover --delay 0.05 --max-paths 500

# Gentle – for anything you do not fully control
python3 edrecon.py -t target --scope scope.yaml \
  --web --discover --delay 1.0 --max-paths 100
```

7. Combined scans

```
# Everything
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --all

# Everything plus content discovery
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --all --discover

# Full assessment, tuned
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
  --all --discover --ports 1-10000 --threads 150 \
  --timeout 1 --delay 0.1 --max-paths 500

# Active only, skipping passive
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --system --web
```

8. Output control

Flag	Effect
(default)	Full teaching layer — all six fields
--no-explain	Observation only, no WHAT/WHY/HOW
--brief	One line per finding
--no-color	Strip ANSI codes (for piping to files)
--no-report	Console only, write no files
--outdir DIR	Change the report directory

```
# Full explanations – use for Labs 1 and 2
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --all

# Brief – use from Lab 3 onward
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --all --brief

# Save console output to a file
python3 edrecon.py -t upes.ac.in --passive-only --no-color \
    > scan_output.txt

# Both screen and file
python3 edrecon.py -t upes.ac.in --passive-only --no-color \
    | tee scan_output.txt

# Per-student report directories
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --all \
    --outdir reports/student_R2142220001

# Console only, no artefacts
python3 edrecon.py -t upes.ac.in --passive-only --no-report
```

9. Reports and evidence

```
# List generated artefacts
ls -lt reports/

# Open the newest HTML report
firefox reports/report_*.html
xdg-open $(ls -t reports/*.html | head -1)

# Pretty-print the JSON report
cat reports/report_*.json | python3 -m json.tool | less

# Verify an evidence chain
python3 edrecon.py --verify-log reports/evidence_192.168.56.101_20260816.jsonl

# Verify the newest log
python3 edrecon.py --verify-log $(ls -t reports/evidence_*.jsonl | head -1)
```

Tampering demonstration (Lab 5):

```
LOG=$(ls -t reports/evidence_*.jsonl | head -1)
python3 edrecon.py --verify-log "$LOG"      # intact
nano "$LOG"                                # change one value on line 3
python3 edrecon.py --verify-log "$LOG"      # chain BROKEN at line 3
```

Extract findings from JSON with jq:

```

R=$(ls -t reports/report_*.json | head -1)

# All high-severity findings
jq '.findings[] | select(.severity=="high") | .title' "$R"

# Severity counts
jq '.summary' "$R"

# Concepts touched
jq '.concepts' "$R"

# ID, severity and title as a table
jq -r '.findings[] | "\(.id)\t\(.severity)\t\(.title)"' "$R" | column -t

# Only confirmed findings
jq '.findings[] | select(.confidence=="confirmed") | .id' "$R"

# Every VERIFY command from the report
jq -r '.findings[].how_to_verify' "$R" | sort -u

```

10. Full flag list

Target

Flag	Default	Meaning
-t, --target	—	Domain, hostname or IP
-s, --scope	—	Path to authorisation scope file
-u, --url	auto	Explicit URL for the web phase

Phases

Flag	Meaning
--passive-only	Passive only. No scope file needed.
--system	Active system scan
--web	Active web scan
--all	All three phases

No phase flag given → passive only. Safe by default.

Scan options

Flag	Default	Meaning
--ports	top	top, 1-1024, or 22,80,443
--threads	50	Concurrent scan threads
--timeout	3.0	Per-connection timeout (seconds)
--delay	0.35	Delay between web requests
--no-banners	off	Skip banner grabbing

Flag	Default	Meaning
--axfr	off	Attempt zone transfer (active)
--discover	off	Content discovery in web phase
--wordlist	built-in	Custom wordlist path
--max-paths	400	Cap on paths probed
--no-whois	off	Skip WHOIS
--no-ct	off	Skip certificate transparency

Output

Flag	Default	Meaning
--brief	off	One line per finding
--no-explain	off	Hide WHAT/WHY/HOW
--outdir	reports	Report directory
--no-color	off	Disable ANSI colour
--no-report	off	Skip file output

Knowledge base and utilities

Flag	Meaning
--kb	Print the whole knowledge base
--kb-search TERM	Search the knowledge base
--concepts	List all concept tags
--verify-log PATH	Verify an evidence hash chain
--version	Version and attribution
--help	Full help text

11. Lab-by-lab command sheet

Lab 1 — Passive reconnaissance

```
python3 edrecon.py --concepts
python3 edrecon.py --kb-search dns-resolution
python3 edrecon.py -t upes.ac.in --passive-only
dig +short TXT upes.ac.in | grep spf1
dig +short TXT _dmarc.upes.ac.in
```

Lab 2 — Scope files and authorisation

```
python3 edrecon.py -t 192.168.56.101 --system
cp scope.example.yaml scope.yaml && nano scope.yaml
sed 's/expires: ./expires: "2020-01-01"/' scope.yaml > /tmp/dead.yaml
python3 edrecon.py -t 192.168.56.101 --system --scope /tmp/dead.yaml
python3 edrecon.py -t scanme.nmap.org --system --scope scope.yaml
python3 edrecon.py --help | grep -i force
```

Lab 3 — Active system scanning

```
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --system
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
  --system --ports 1-10000 --threads 100 --timeout 1
nmap -sV -p- 192.168.56.101 # compare
python3 edrecon.py --kb-search banner-grabbing
```

Lab 4 — Web scanning

```
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --web
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \
  --web --discover --delay 0.1
python3 edrecon.py --kb-search baseline-comparison
firefox reports/report_*.html
```

Lab 5 — Evidence integrity

```
LOG=$(ls -t reports/evidence_*.jsonl | head -1)
python3 edrecon.py --verify-log "$LOG"
head -3 "$LOG" | python3 -m json.tool
nano "$LOG"
python3 edrecon.py --verify-log "$LOG"
```

Lab 6 — Extending the tool

```
nano knowledge/web.yaml # add an entry
python3 edrecon.py --kb-search YOUR-ID # confirm it loads
python3 -m py_compile edrecon.py core/*.py modules/*.py
```

12. Maintenance and troubleshooting

```
# Confirm all modules compile
python3 -m py_compile edrecon.py core/*.py modules/**/*.py && echo OK

# Check which dependencies are present
python3 -c "
for m in ['yaml','requests','dns.resolver','bs4','whois']:
    try:
        __import__(m); print(f'{m:16} OK')
    except ImportError:
        print(f'{m:16} MISSING')
"

# List installed packages in the venv
pip list

# Reinstall dependencies
pip install -r requirements.txt --force-reinstall

# Fix a root-owned virtual environment
sudo chown -R $USER:$USER ~/edrecon
rm -rf .venv && python3 -m venv .venv
source .venv/bin/activate && pip install -r requirements.txt

# Clear caches and old reports
find . -name "__pycache__" -type d -exec rm -rf {} + 2>/dev/null
rm -f reports/*

# Archive a student's results
tar czf student_R2142220001_results.tar.gz reports/
```

Common errors:

Symptom	Cause	Fix
ModuleNotFoundError	venv not activated	source .venv/bin/activate
Permission denied creating venv	directory root-owned	sudo chown -R \$USER:\$USER .
uninstall-no-record-file	ran sudo pip	recreate venv, install without sudo
AUTHORISATION EXPIRED	scope lapsed	update expires — only if still authorised
TARGET NOT AUTHORISED	not in scope	add it only with written permission
crt.sh HTTP 429	rate limited	wait, or use --no-ct
Content discovery finds nothing	soft-404 filtering	check output for the baseline note

EDRecon — an EDCatalyst teaching tool. Dr. Keshav Sinha, UPES, Dehradun, India. Authorised testing only.

Appendix A - Knowledge Base

Every explanation EDRecon can emit. These live in `knowledge/*.yaml` and are fully decoupled from scanner logic - edit, translate or extend them without touching Python.

DNS, Email Authentication and Registration

[DNS-001] A/AAAA records resolved

WHAT	The domain maps to one or more IP addresses. A records are IPv4, AAAA records are IPv6. This is the address the client actually connects to after resolution.
WHY	This is the first pivot in any assessment. The IP tells you who hosts the target (cloud, on-premise, CDN), which netblock it sits in, and therefore what else the same owner may run nearby. If AAAA exists but firewall rules were only written for IPv4, the IPv6 path is often an unmonitored way in.
HOW	A recursive DNS query to your configured resolver. No packet was sent to the target itself - the resolver did the work, which is why this counts as passive reconnaissance.
VERIFY	<code>dig +short A example.com ; dig +short AAAA example.com</code>
FIX	Not a weakness. If the address should not be public, place the service behind a proxy or CDN so the origin IP is not directly resolvable.
META	severity: info RFC 1035 . RFC 3596 tags: dns-resolution . recursive-query . ipv4-ipv6

[DNS-002] MX records disclose mail infrastructure

WHAT	Mail Exchanger records name the hosts that accept email for this domain, with a preference value setting their order.
WHY	MX records reveal whether mail is self-hosted or outsourced. That single fact shapes an entire phishing strategy: a Google Workspace or Microsoft 365 tenant implies a known login page to clone, while a self-hosted server implies a mail daemon you can fingerprint and a possible open relay to test.
HOW	Recursive DNS query for the MX record type.
VERIFY	<code>dig +short MX example.com</code>
FIX	MX records must be public for mail to work. Reduce the value of the disclosure by enforcing SPF, DKIM and DMARC so a spoofed sender is rejected regardless.
META	severity: info RFC 5321 tags: dns-resolution . mail-infrastructure . phishing-surface

[DNS-003] SPF record present

WHAT	A TXT record beginning <code>v=spf1</code> lists which servers are permitted to send mail on behalf of this domain.
WHY	SPF also leaks infrastructure. An include: directive names every third-party service authorised to send as this domain - CRM, ticketing, marketing platforms - each of which is a supply-chain path in. The final mechanism matters too: <code>-all</code> is a hard fail, <code>~all</code> is a soft fail that many receivers still deliver.
HOW	TXT record lookup, filtered for the <code>v=spf1</code> prefix.
VERIFY	<code>dig +short TXT example.com grep spf1</code>
FIX	End the record with <code>-all</code> rather than <code>~all</code> once you are confident the list is complete. Keep the include: list minimal and audited; the lookup limit is 10 DNS queries.
META	severity: info RFC 7208 tags: email-authentication . spf . supply-chain

[DNS-004] No SPF record found

WHAT	The domain publishes no v=spf1 TXT record, so receivers have no authoritative list of permitted senders.
WHY	Without SPF, anyone can send email claiming to be from this domain and receivers have no DNS-based reason to reject it. For a university or enterprise domain this is a direct, low-effort phishing enabler against staff and students.
HOW	TXT lookup returned no record with the v=spf1 prefix.
VERIFY	<code>dig +short TXT example.com</code>
FIX	Publish an SPF record listing legitimate senders, then move to -all after a monitoring period. Pair with DKIM and a DMARC policy.
META	severity: medium RFC 7208 . CWE-290 tags: email-authentication . spf . spoofing

[DNS-005] DMARC policy published

WHAT	A TXT record at <code>_dmarc.<domain></code> tells receivers what to do when SPF and DKIM checks fail, and where to send aggregate reports.
WHY	The <code>p=</code> value is what actually matters. <code>p=none</code> only monitors and blocks nothing, so a domain can appear protected while remaining fully spoofable. <code>p=quarantine</code> or <code>p=reject</code> is where real protection begins.
HOW	TXT lookup against the <code>_dmarc</code> subdomain.
VERIFY	<code>dig +short TXT _dmarc.example.com</code>
FIX	Progress from <code>p=none</code> to <code>p=quarantine</code> to <code>p=reject</code> as report data confirms legitimate senders are aligned.
META	severity: info RFC 7489 tags: email-authentication . dmarc . policy-enforcement

[DNS-006] No DMARC record found

WHAT	No policy exists at <code>_dmarc.<domain></code> , so receivers fall back to their own heuristics when authentication fails.
WHY	SPF and DKIM without DMARC leave a gap: neither covers the From header the user actually sees. DMARC ties authentication to that visible header. Its absence means display-name spoofing has no policy-level defence.
HOW	TXT lookup against <code>_dmarc</code> returned nothing.
VERIFY	<code>dig +short TXT _dmarc.example.com</code>
FIX	Publish a record starting at <code>p=none</code> with <code>rua=</code> reporting, review the reports, then tighten the policy.
META	severity: medium RFC 7489 tags: email-authentication . dmarc . spoofing

[DNS-007] Nameservers identified

WHAT	NS records name the authoritative DNS servers for the zone.
WHY	Nameservers identify the DNS provider and are themselves targets. A dangling NS delegation, an expired provider account, or a misconfigured server accepting zone transfers all lead to full control of the domain's resolution.
HOW	Recursive query for the NS record type.
VERIFY	<code>dig +short NS example.com</code>
FIX	Use at least two providers, restrict zone transfers, and audit delegations for nameservers you no longer control.
META	severity: info RFC 1035 tags: dns-resolution . authoritative-dns . delegation

[DNS-008] Zone transfer (AXFR) permitted

WHAT	A nameserver returned the full contents of the DNS zone in response to an unauthenticated AXFR request.
WHY	This is the single most valuable DNS misconfiguration. Instead of guessing subdomains one at a time, an attacker receives the complete list - including internal hostnames, staging servers, VPN endpoints and development boxes that were never meant to be discoverable. Reconnaissance that would take hours completes in one query.
HOW	An AXFR query was sent to each authoritative nameserver. A correctly configured server refuses this from unauthorised clients.
VERIFY	<code>dig AXFR example.com @ns1.example.com</code>
FIX	Restrict AXFR to known secondary servers by IP, and prefer TSIG keys for authentication. In BIND, use <code>allow-transfer</code> with an explicit ACL.
META	severity: high RFC 5936 . CWE-200 tags: zone-transfer . axfr . information-disclosure

[DNS-009] TXT records disclose service verification tokens

WHAT	TXT records often hold ownership-verification strings for third-party SaaS platforms.
WHY	Each token names a service the organisation uses. Collectively they map the SaaS footprint - which is both a phishing pretext list and a set of accounts whose compromise affects the domain.
HOW	TXT record lookup, excluding SPF and DMARC entries.
VERIFY	<code>dig +short TXT example.com</code>
FIX	Remove verification tokens for services no longer in use; stale tokens can allow domain re-claim on abandoned platforms.
META	severity: low CWE-200 tags: information-disclosure . saas-footprint

[DNS-010] Subdomain discovered

WHAT	A hostname under the target domain that resolves to an address.
WHY	Subdomains multiply attack surface. Development, staging and legacy hosts are patched less often than the main site and frequently run with debug features enabled. A subdomain pointing at a deprovisioned cloud resource is vulnerable to takeover.
HOW	Aggregated from certificate transparency logs and DNS resolution. CT logs are public records of every certificate issued, so this is passive - no request reached the target.
VERIFY	<code>curl -s 'https://crt.sh/?q=%25.example.com&output=json'</code>
FIX	Maintain a DNS inventory, remove records for decommissioned services immediately, and consider whether internal hostnames need public DNS.
META	severity: info RFC 6962 tags: subdomain-enumeration . certificate-transparency . attack-surface

[WHO-001] WHOIS registration data retrieved

WHAT	Registration metadata for the domain: registrar, creation and expiry dates, and contact details where not redacted.
WHY	Creation date distinguishes an established domain from one registered last week for a phishing campaign. Expiry date flags a domain at risk of lapsing and being re-registered by someone else. Unredacted contact details give names and emails for social engineering.
HOW	A WHOIS query to the registry, not the target. Fully passive.
VERIFY	<code>whois example.com</code>
FIX	Enable registrar privacy/proxy services, enable registrar lock, and set auto-renew to prevent accidental expiry.
META	severity: info RFC 3912 . RFC 9083 tags: whois . domain-registration . osint

[WHO-002] Domain expires soon

WHAT	The registration expiry date is within 30 days.
WHY	An expired domain can be registered by anyone. Whoever holds it controls email, DNS and any service that trusts the domain - including password resets sent to addresses at that domain.
HOW	Expiry field parsed from the WHOIS response.
VERIFY	<code>whois example.com grep -i expir</code>
FIX	Enable auto-renew and registrar lock; monitor expiry centrally.
META	severity: medium CWE-350 tags: whois . domain-expiry . takeover-risk

Hosts, Ports and Services

[HOST-001] Host is responsive

WHAT	The target responded to a probe, confirming a live system at this address at this moment.
WHY	Liveness is the gate for everything else. Note the reasoning is one-directional: a response proves the host is up, but silence does not prove it is down - it may simply be dropping the probe type you used. Students who forget this write off entire live subnets.
HOW	A TCP connection attempt to a common port. Unlike ICMP ping, this works through firewalls that drop echo requests but must still permit traffic to published services.
VERIFY	<code>nmap -sn 192.168.56.101</code>
FIX	Not a weakness. If the host should not be reachable from this network segment, review firewall and segmentation rules.
META	severity: info RFC 793 tags: host-discovery . tcp-handshake . negative-evidence

[PORT-001] Open TCP port

WHAT	A full three-way handshake completed on this port, so a service is listening and accepting connections.
WHY	Every open port is an entry point. The question is never just "what is open" but "why is it reachable from here" - a database port open to the internet is a very different finding from the same port open only on a management VLAN.
HOW	TCP connect scan: SYN sent, SYN-ACK received, ACK sent, then closed. Because the handshake completes, the connection is logged by the target - this is a noisy scan by design, and that visibility is the honest trade-off for not needing root privileges.
VERIFY	<code>nmap -sT -p <port> <target></code>
FIX	Close unused services, bind them to localhost or a management interface, and enforce host-based firewall rules.
META	severity: info RFC 793 tags: port-scanning . tcp-handshake . attack-surface

[PORT-002] Service banner captured

WHAT	The service voluntarily disclosed its identity and often its exact version on connection.
WHY	A version string converts a generic open port into a searchable vulnerability query. This is the pivot from enumeration to exploitation: software name plus version maps directly onto CVE databases and exploit modules. Banners can also be edited by the administrator, so treat them as claims rather than proof.
HOW	Connected to the port and read the first bytes the service sent without being asked. Many protocols greet the client by design.
VERIFY	<code>nc -nv <target> <port> # or: nmap -sV -p <port> <target></code>
FIX	Suppress or genericise version strings where the daemon permits. Understand this is cosmetic - patching the actual software is the real fix, since fingerprinting can identify versions without banners.
META	severity: low CWE-200 tags: banner-grabbing . service-fingerprinting . cve-mapping

[PORT-003] Cleartext protocol exposed

WHAT	A service was found on a port whose protocol transmits credentials and data without encryption.
WHY	Anyone positioned on the network path - a compromised switch, a rogue access point, an ARP-spoofing host on the same LAN - can read the credentials directly off the wire. No cryptographic attack is needed; the data is simply in plaintext.
HOW	Port number matched a known cleartext protocol and a connection succeeded.
VERIFY	Capture the session in Wireshark and follow the TCP stream.
FIX	Replace with the encrypted equivalent: SSH for Telnet, FTPS/SFTP for FTP, IMAPS/POP3S for their plain forms, and disable the legacy listener entirely rather than merely adding the secure one.
META	severity: high CWE-319 tags: cleartext-protocol . credential-exposure . network-sniffing

[PORT-004] Database port reachable

WHAT	A port associated with a database engine accepted a connection from the scanning host.
WHY	Databases are meant to be reached by application servers, not by arbitrary hosts. Direct reachability suggests missing segmentation. Many database engines historically shipped with blank or default credentials, making an exposed port a short path to the entire data store.
HOW	TCP connect succeeded on a well-known database port.
VERIFY	<code>nmap -sV -p 3306,5432,1433,27017,6379 <target></code>
FIX	Bind the listener to localhost or a private interface, require authentication, and restrict access with firewall rules to application hosts only.
META	severity: high CWE-668 . CWE-1188 tags: network-segmentation . database-exposure . default-credentials

[PORT-005] Remote management port reachable

WHAT	A port used for remote administration is accepting connections.
WHY	Management interfaces are the highest-value targets on any host: success there means interactive control, not just data access. When exposed broadly they attract continuous credential-stuffing traffic.
HOW	TCP connect succeeded on a known management port.
VERIFY	<code>nmap -sV -p 22,23,3389,5900 <target></code>
FIX	Restrict to a management network or VPN, enforce key-based or multi-factor authentication, and rate-limit authentication attempts.
META	severity: medium CWE-284 tags: remote-administration . access-control . credential-attacks

[PORT-006] Filtered or closed port behaviour observed

WHAT	The port did not complete a handshake. A refusal (RST) means closed; a timeout usually means filtered by a firewall.
WHY	The distinction is diagnostically important. A closed port tells you the host is alive but nothing listens there. A silent drop tells you a filtering device sits in the path - which itself maps the network's defensive posture.
HOW	Connection attempt returned RST (closed) or timed out (filtered).
VERIFY	<code>nmap -sT --reason -p <port> <target></code>
FIX	Default-deny filtering is correct. Prefer dropping over rejecting for unused ports so that scanners receive no confirmation at all.
META	severity: info RFC 793 tags: port-states . firewall-behaviour . tcp-reset

Certificates and TLS Configuration

[TLS-001] TLS certificate retrieved

WHAT	The server presented an X.509 certificate during the handshake, binding a public key to the identity claimed in the subject and subject alternative name fields.
WHY	The certificate is a free intelligence document. SAN entries often list every hostname the server answers for - including internal names, staging environments and sibling domains the operator forgot were on the same certificate. The issuer reveals the CA and whether the deployment is automated.
HOW	A TLS handshake was initiated and the server's certificate chain read from the ServerHello. No authentication or data exchange followed.
VERIFY	<code>openssl s_client -connect host:443 -servername host < /dev/null</code>
FIX	Avoid listing internal hostnames in public certificates; use separate certificates for internal and external names.
META	severity: info RFC 5280 . RFC 8446 tags: tls-handshake . x509 . certificate-chain

[TLS-002] Certificate has expired

WHAT	The current date is past the notAfter field, so the certificate is no longer valid.
WHY	Clients show interstitial warnings, which trains users to click through security errors - the exact habit that makes real interception attacks succeed later. It also signals unmonitored infrastructure, which correlates with missing patches elsewhere.
HOW	notAfter field compared against the current UTC date.
VERIFY	<code>openssl s_client -connect host:443 < /dev/null openssl x509 -noout -dates</code>
FIX	Automate renewal (ACME/certbot) and alert on certificates within 30 days of expiry.
META	severity: high RFC 5280 . CWE-298 tags: certificate-validity . expiry . user-conditioning

[TLS-003] Certificate expires soon

WHAT	The notAfter date is within 30 days.
WHY	A predictable outage. If renewal is manual, this is the window in which it is forgotten.
HOW	notAfter field compared against current date plus 30 days.
VERIFY	<code>openssl s_client -connect host:443 < /dev/null openssl x509 -noout -enddate</code>
FIX	Automate renewal and monitoring.
META	severity: low RFC 5280 tags: certificate-validity . expiry

[TLS-004] Hostname does not match certificate

WHAT	The hostname connected to does not appear in the certificate's common name or subject alternative names.
WHY	Certificate validation exists to prove you are talking to the right server. A mismatch breaks that proof - a client that proceeds anyway has encryption without authentication, which stops passive eavesdropping but not an active machine-in-the-middle.
HOW	The requested hostname was compared against the CN and SAN entries, with wildcard rules applied.
VERIFY	<code>openssl s_client -connect host:443 -servername host < /dev/null openssl x509 -noout -text grep -A1 'Subject Alternative Name'</code>
FIX	Reissue the certificate including every hostname served, or correct the virtual host configuration so the right certificate is presented.
META	severity: high RFC 6125 . CWE-297 tags: hostname-verification . san . mitm

[TLS-005] Self-signed certificate

WHAT	The certificate's issuer equals its subject - it was not signed by an independent certificate authority.
WHY	There is no third party vouching for the identity, so a client cannot distinguish the real server from an impostor presenting a different self-signed certificate. Acceptable on an isolated lab host; unacceptable on anything users reach.
HOW	Issuer and subject distinguished names compared for equality.
VERIFY	<code>openssl s_client -connect host:443 < /dev/null openssl x509 -noout -issuer -subject</code>
FIX	Use a publicly trusted CA for external services, or an internal PKI whose root is distributed to managed clients.
META	severity: medium RFC 5280 . CWE-295 tags: certificate-authority . trust-chain . self-signed

[TLS-006] Obsolete TLS protocol version supported

WHAT	The server negotiated SSLv3, TLS 1.0 or TLS 1.1 - all formally deprecated.
WHY	These versions carry structural weaknesses that cannot be patched away: SSLv3 is broken by POODLE, TLS 1.0 by BEAST and its CBC handling. Supporting them enables downgrade attacks in which an active attacker forces the weakest mutually supported version.
HOW	Separate handshakes were attempted, each pinned to a specific protocol version, to see which the server accepts.
VERIFY	<code>openssl s_client -connect host:443 -tls1_1 < /dev/null</code>
FIX	Disable everything below TLS 1.2 and prefer TLS 1.3. In nginx: <code>ssl_protocols TLSv1.2 TLSv1.3;</code>
META	severity: medium RFC 8996 . CWE-327 tags: protocol-downgrade . deprecated-crypto . poodle-beast

[TLS-007] Modern TLS supported

WHAT	The server negotiated TLS 1.2 or TLS 1.3.
WHY	Confirms the baseline is met. TLS 1.3 additionally removes legacy cipher suites, mandates forward secrecy, and encrypts more of the handshake itself.
HOW	Version-pinned handshake succeeded.
VERIFY	<code>openssl s_client -connect host:443 -tls1_3 < /dev/null</code>
FIX	No action required.
META	severity: info RFC 8446 tags: tls-handshake . forward-secrecy

[TLS-008] Weak cipher suite negotiated

WHAT	The agreed cipher suite uses an algorithm considered weak - such as RC4, 3DES, or a suite offering no forward secrecy.
WHY	Without forward secrecy, an attacker who records traffic today and obtains the server's private key later can decrypt all of it retrospectively. Weak ciphers reduce the work factor of that attack from impractical to merely expensive.
HOW	The cipher name from the completed handshake was matched against a deny list.
VERIFY	<code>nmap --script ssl-enum-ciphers -p 443 <target></code>
FIX	Restrict to AEAD suites with ECDHE key exchange. Remove RC4, 3DES, export and NULL suites entirely.
META	severity: medium RFC 7465 . CWE-327 tags: cipher-suites . forward-secrecy . retrospective-decryption

[TLS-009] Certificate SAN entries disclose additional hostnames

WHAT	The subject alternative name extension lists multiple hostnames covered by this certificate.
WHY	Each additional name is a reconnaissance lead that required no scanning to obtain. Shared certificates across environments often expose internal or pre-production hostnames alongside the public one.
HOW	SAN extension parsed from the presented certificate.
VERIFY	<code>openssl s_client -connect host:443 < /dev/null openssl x509 -noout -text grep DNS:</code>
FIX	Issue separate certificates per environment so internal names never appear in a publicly reachable certificate.
META	severity: low RFC 5280 . CWE-200 tags: san . information-disclosure . attack-surface

HTTP, Headers and Web Surface

[HDR-001] Strict-Transport-Security header missing

WHAT	The server does not instruct browsers to use HTTPS exclusively for future requests.
WHY	Without HSTS, a user typing the bare hostname makes an initial plain HTTP request. An attacker on the path can intercept that first request and strip the redirect, keeping the session on HTTP indefinitely - the classic SSL-stripping attack. HSTS closes the window after the first successful visit.
HOW	Response headers checked for Strict-Transport-Security.
VERIFY	<code>curl -sI https://<target>/ grep -i strict-transport</code>
FIX	Send: Strict-Transport-Security: max-age=31536000; includeSubDomains Add preload only after confirming every subdomain supports HTTPS.
META	severity: medium RFC 6797 . CWE-319 tags: hsts . ssl-stripping . transport-security

[HDR-002] Content-Security-Policy header missing

WHAT	No policy restricts which sources the browser may load scripts, styles or frames from.
WHY	CSP is the main defence-in-depth control against cross-site scripting. Without it, any successful injection executes with the full privileges of the page. With a strict policy, many injections are neutralised even when the underlying flaw exists.
HOW	Response headers checked for Content-Security-Policy.
VERIFY	<code>curl -sI https://<target>/ grep -i content-security</code>
FIX	Start in report-only mode to find violations, then enforce. Avoid unsafe-inline and unsafe-eval; prefer nonces or hashes.
META	severity: medium CWE-1021 . OWASP A03:2021 tags: csp . xss-mitigation . defence-in-depth

[HDR-003] X-Frame-Options / frame-ancestors missing

WHAT	Nothing prevents the page from being embedded in a frame on another site.
WHY	Enables clickjacking: the attacker overlays an invisible frame of this site above their own interface, so a user who thinks they are clicking a harmless button is actually actioning a request on this application while fully authenticated.
HOW	Checked for X-Frame-Options and for a frame-ancestors directive inside any CSP header.
VERIFY	<code>curl -sI https://<target>/ grep -iE 'x-frame frame-ancestors'</code>
FIX	Prefer CSP frame-ancestors 'self'; retain X-Frame-Options: DENY for older browsers.
META	severity: medium RFC 7034 . CWE-1021 tags: clickjacking . framing-protection

[HDR-004] X-Content-Type-Options missing

WHAT	The browser is permitted to MIME-sniff responses rather than trusting the declared Content-Type.
WHY	An uploaded file served with a benign content type may still be interpreted as script if the browser sniffs its contents, turning a file upload feature into stored cross-site scripting.
HOW	Response headers checked for X-Content-Type-Options.
VERIFY	<code>curl -sI https://<target>/ grep -i x-content-type</code>
FIX	Send: X-Content-Type-Options: nosniff
META	severity: low CWE-430 tags: mime-sniffing . content-type

[HDR-005] Cookie set without Secure or HttpOnly flag

WHAT	A Set-Cookie header omitted the Secure flag, the HttpOnly flag, or both.
WHY	Without Secure, the cookie is transmitted over plain HTTP and can be captured on the network path. Without HttpOnly, JavaScript can read it - so any cross-site scripting flaw escalates directly into session theft. For a session cookie this is the difference between an XSS that defaces a page and one that takes over accounts.
HOW	Set-Cookie headers parsed and flag attributes checked.
VERIFY	<code>curl -sI https://<target>/ grep -i set-cookie</code>
FIX	Set Secure, HttpOnly and SameSite=Lax or Strict on all session cookies.
META	severity: medium RFC 6265 . CWE-1004 . CWE-614 tags: cookie-security . session-hijacking . xss-escalation

[HDR-006] Security headers correctly configured

WHAT	The expected security header was present with a sound value.
WHY	Worth recording explicitly. A report that only lists failures teaches students what is broken but not what correct configuration looks like.
HOW	Header present and value validated.
VERIFY	<code>curl -sI https://<target>/</code>
FIX	No action required.
META	severity: info OWASP Secure Headers Project tags: security-headers . baseline-configuration

[HTTP-001] HTTP service responded

WHAT	The server returned an HTTP response, confirming a web service and revealing its status code and response headers.
WHY	The response is the entry point for all web testing. Status codes, redirects and header set together indicate the technology stack, whether a reverse proxy sits in front, and how the application handles unknown requests.
HOW	A single HTTP GET request to the root path.
VERIFY	<code>curl -I http://<target>/</code>
FIX	Not a weakness.
META	severity: info RFC 9110 tags: http-protocol . status-codes

[HTTP-002] Server header discloses software and version

WHAT	The Server response header names the web server software, often with its exact version number.
WHY	Same principle as a service banner: a version string is a direct lookup key into vulnerability databases. It also tells an attacker which exploitation path is worth attempting first, saving them the noisy trial-and-error that defenders are most likely to detect.
HOW	Read from the Server header of the HTTP response.
VERIFY	<code>curl -sI http://<target>/ grep -i '^server:'</code>
FIX	Apache: ServerTokens Prod and ServerSignature Off. nginx: server_tokens off; Recognise this is obfuscation, not remediation - patch the software.
META	severity: low CWE-200 tags: information-disclosure . version-disclosure . fingerprinting

[HTTP-003] Technology stack disclosed via headers

WHAT	Headers such as X-Powered-By, X-AspNet-Version or X-Generator name the application framework or runtime.
WHY	Framework and version narrow the vulnerability search dramatically and reveal which default paths, error pages and known misconfigurations to probe next.
HOW	Response headers matched against a list of disclosure headers.
VERIFY	<code>curl -sI http://<target>/</code>
FIX	Remove these headers at the application or reverse proxy layer. PHP: <code>expose_php = Off</code> . nginx: <code>proxy_hide_header X-Powered-By</code> ;
META	severity: low CWE-200 tags: information-disclosure . framework-fingerprinting

[WEB-001] HTTP does not redirect to HTTPS

WHAT	A plain HTTP request was served content directly instead of being redirected to the encrypted equivalent.
WHY	All traffic, including any credentials submitted, travels in cleartext. Even if an HTTPS version exists, users and links that reach the HTTP endpoint stay unencrypted.
HOW	HTTP request to port 80 returned 200 rather than a 301/302 to https://.
VERIFY	<code>curl -sI http://<target>/</code>
FIX	Return a 301 to the HTTPS URL for all HTTP requests, then add HSTS.
META	severity: medium CWE-319 tags: transport-security . redirect . cleartext

[WEB-002] Interesting path discovered

WHAT	A request to this path returned a response indicating the resource exists.
WHY	Administrative panels, backup files, version control directories and configuration files are frequently reachable without authentication because they were never intended to be requested directly. Content discovery finds what is present but unlinked.
HOW	Wordlist-driven requests, rate-limited, with the status code and response length compared against a baseline for a path known not to exist. That baseline is what distinguishes a real hit from a server that returns 200 for everything.
VERIFY	<code>curl -sI http://<target>/<path></code>
FIX	Remove files that should not be deployed, require authentication on administrative paths, and block access to .git, .env and backup extensions at the web server.
META	severity: medium CWE-425 . CWE-548 tags: content-discovery . forced-browsing . baseline-comparison

[WEB-003] Directory listing enabled

WHAT	The server returned an index of files in a directory instead of a document or an error.
WHY	Removes all guesswork from content discovery. Backups, source archives and configuration files that would never be found by a wordlist are simply listed.
HOW	Response body matched directory index signatures.
VERIFY	<code>curl -s http://<target>/<path>/ head</code>
FIX	Apache: Options -Indexes. nginx: autoindex off;
META	severity: medium CWE-548 tags: directory-listing . information-disclosure

[WEB-004] robots.txt discloses paths

WHAT	The robots.txt file lists paths the operator asked crawlers to avoid.
WHY	A useful inversion: paths listed as disallowed are frequently the ones the operator considers sensitive. The file is a public, voluntarily-published hint list, and reading it sends one ordinary request.
HOW	Fetches /robots.txt and parsed Disallow entries.
VERIFY	<code>curl -s http://<target>/robots.txt</code>
FIX	Never rely on robots.txt for security. Use authentication and noindex headers; keep genuinely sensitive paths out of the file.
META	severity: low CWE-200 tags: robots-txt . information-disclosure . security-by-obscurity

[WEB-005] Web form discovered

WHAT	An HTML form with its method, action and input fields was found on the page.
WHY	Forms are where user input enters the application, making them the starting point for input-validation testing. The absence of a CSRF token field on a state-changing POST form is itself a finding.
HOW	Response HTML parsed for form elements and their inputs.
VERIFY	View page source and inspect the <form> element.
FIX	Include anti-CSRF tokens on state-changing forms, validate all input server-side, and mark password fields with the correct input type.
META	severity: info CWE-352 . OWASP A01:2021 tags: form-analysis . csrf . input-surface

[WEB-006] Password form submitted over HTTP

WHAT	A form containing a password input posts to a plain HTTP URL.
WHY	Credentials are transmitted in cleartext and readable by anyone on the network path. This is among the most direct credential-exposure findings possible in a web assessment.
HOW	Form inputs scanned for type=password, with the resolved action URL scheme checked.
VERIFY	Inspect the form action attribute in page source.
FIX	Serve the page and its action target over HTTPS only, and add HSTS.
META	severity: high CWE-319 . CWE-522 tags: credential-exposure . cleartext . form-analysis

[WEB-007] Version control directory exposed

WHAT	A path such as /.git/ or /.svn/ is reachable over HTTP.
WHY	An exposed repository often allows reconstruction of the entire application source, including commit history. History is the real prize: credentials and keys that were committed and later removed remain recoverable from earlier commits.
HOW	Request to the version control path returned an existing resource.
VERIFY	<code>curl -sI http://<target>/.git/HEAD</code>
FIX	Deny access to dot-directories at the web server, and deploy build artefacts rather than working copies. Rotate any credential that was ever committed.
META	severity: high CWE-527 . CWE-540 tags: source-disclosure . version-control . secret-leakage

Part IV - Complete Source Code

The full implementation follows, in dependency order. Each file is preceded by a note on its role. Line lengths are wrapped to fit the page; the shipped source is unwrapped.

File	Lines	Role
edrecon.py	481	Main CLI entry point. Argument parsing, phase orchestration, scope enf
core/finding.py	167	The Finding dataclass - the teaching layer schema. Every module return
core/scope_guard.py	230	Authorisation enforcement. Loads the scope file and gates every active
core/knowledge.py	70	Loads the YAML knowledge base that supplies all explanatory text.
core/evidence.py	110	Rate limiting and the hash-chained evidence log.
core/output.py	354	Console and HTML renderers.
core/banner.py	121	Banner and ANSI colour handling.
modules/passive/dns_recon.py	236	PASSIVE - DNS, SPF, DMARC, and the AXFR check.
modules/passive/whois_ct.py	164	PASSIVE - WHOIS registration and certificate transparency subdomain ha
modules/system/port_scan.py	294	ACTIVE - host discovery, TCP connect scan, banner grabbing.
modules/system/tls_audit.py	234	ACTIVE - certificate and TLS configuration audit.
modules/web/http_audit.py	272	ACTIVE - HTTP fingerprinting, security headers, cookies, forms.
modules/web/content_discovery.py	228	ACTIVE - wordlist path discovery with baseline comparison.
scope.example.yaml	67	Scope file template. Copy and edit for your lab.
requirements.txt	13	Python dependencies.

edrecon.py

Main CLI entry point. Argument parsing, phase orchestration, scope enforcement, and report generation.

```
#!/usr/bin/env python3
"""
EDRecon - Explainable Reconnaissance Framework
=====

An EDCatalyst teaching tool.
Dr. Keshav Sinha, UPES, Dehradun, India.

Every finding explains WHAT was observed, WHY it matters, and HOW it was
detected -- because a scanner that only prints results teaches students to
read output, not to reason about it.

AUTHORISED USE ONLY. Active modules require a valid scope file.
"""

import argparse
import os
import sys
import time
from datetime import datetime
from urllib.parse import urlparse

HERE = os.path.dirname(os.path.abspath(__file__))
if HERE not in sys.path:
    sys.path.insert(0, HERE)

from core import output as out
from core.banner import C, VERSION, show_banner
from core.evidence import EvidenceLog, RateLimiter
from core.finding import FindingSet, CONFIDENCE_MEANING
from core.knowledge import kb
from core.scope_guard import ScopeGuard, ScopeError, OutOfScope

from modules.passive import dns_recon
from modules.passive import whois_ct
from modules.system import port_scan
from modules.system import tls_audit
from modules.web import http_audit
from modules.web import content_discovery

# -----
# Module runner -- isolates failures so one broken module never ends the run
# -----

def run_module(fs, name, label, fn, *args, **kwargs):
    out.step(label)
    try:
        result = fn(*args, **kwargs)
    except OutOfScope as exc:
        out.error(str(exc))
        fs.record_error(name, "out of scope")
        return None
    except RuntimeError as exc:
        out.warn("{}: {}".format(name, exc))
        fs.record_error(name, str(exc))
        return None
    except KeyboardInterrupt:
        raise
    except Exception as exc:
        out.warn("{} failed: {}: {}".format(name, type(exc).__name__, exc))
        fs.record_error(name, "{}: {}".format(type(exc).__name__, exc))
        return None
    return result

def emit(fs, findings, brief, explain):
    if not findings:
        out.info("No findings from this module.")
        return
    fs.extend(findings)
    for f in findings:
        out.render_finding(f, brief=brief, explain=explain)

# -----
# Phases
# -----
```

```

def phase_passive(fs, target, args, rate, elog, scope):
    out.section("PHASE 1 - PASSIVE RECONNAISSANCE")
    out.info(
        "No packets are sent to the target in this phase. Queries go to "
        "resolvers,\n registries and public logs, so the target's own "
        "logs stay empty."
    )

    res = run_module(fs, "dns_recon", "DNS records", dns_recon.run,
                    target, scope=scope, evidence_log=elog,
                    opts={"try_axfr": args.axfr, "timeout": args.timeout})
    emit(fs, res, args.brief, not args.no_explain)

    if not args.no_whois:
        res = run_module(fs, "whois_lookup", "WHOIS registration",
                        whois_ct.run_whois, target, evidence_log=elog)
        emit(fs, res, args.brief, not args.no_explain)

    subdomains = []
    if not args.no_ct:
        res = run_module(fs, "cert_transparency",
                        "Certificate transparency logs (crt.sh)",
                        whois_ct.run_ct, target, rate=rate, evidence_log=elog,
                        opts={"timeout": args.timeout + 15})

        if res:
            findings, subdomains = res
            emit(fs, findings, args.brief, not args.no_explain)
            if subdomains and not args.brief:
                out.info("Hostnames recovered ({}):".format(len(subdomains)))
                for s in subdomains[:30]:
                    print(C.GREY + "      " + s + C.RESET)
                if len(subdomains) > 30:
                    print(C.GREY + "      ... and {} more (see report)".format(
                        len(subdomains) - 30) + C.RESET)
    return subdomains

def phase_system(fs, host, args, elog, scope):
    out.section("PHASE 2 - ACTIVE SYSTEM SCAN")
    out.warn(
        "Packets are now being sent to {}. This is logged by the target.".format(host)
    )

    res = run_module(fs, "host_discovery", "Host discovery",
                    port_scan.discover, host, scope,
                    timeout=args.timeout, evidence_log=elog)

    if res is None:
        return []
    findings, alive = res
    emit(fs, findings, args.brief, not args.no_explain)

    if not alive:
        out.warn(
            "No response on common ports. The host may be down, or filtering "
            "all probes.\n Remember: silence is not proof of absence."
        )
        return []

    ports = port_scan.parse_ports(args.ports)
    out.info("Probing {} port(s) with {} threads...".format(len(ports), args.threads))

    res = run_module(fs, "port_scan", "TCP connect scan", port_scan.scan,
                    host, scope, ports=ports, timeout=args.timeout,
                    threads=args.threads, banners=not args.no_banners,
                    evidence_log=elog)

    if res is None:
        return []
    findings, open_ports = res
    emit(fs, findings, args.brief, not args.no_explain)

    # TLS on any port that looks encrypted
    tls_ports = [p for p in open_ports if p in (443, 8443, 993, 995, 465, 636)]
    for port in tls_ports:
        res = run_module(fs, "tls_audit", "TLS audit on port {}".format(port),
                        tls_audit.audit, host, scope, port=port,
                        timeout=args.timeout + 4, evidence_log=elog)
        emit(fs, res, args.brief, not args.no_explain)

```

```

return open_ports

def phase_web(fs, host, open_ports, args, rate, elog, scope):
    out.section("PHASE 3 - ACTIVE WEB SCAN")

    candidates = []
    if args.url:
        candidates.append(args.url)
    else:
        web_ports = [p for p in (open_ports or []) if p in port_scan.WEB_PORTS]
        if not web_ports:
            web_ports = [80, 443]
        for p in web_ports:
            scheme = "https" if p in (443, 8443) else "http"
            suffix = "" if p in (80, 443) else ":{0}".format(p)
            candidates.append("{0}://{0}{1}/".format(scheme, host, suffix))

    seen = set()
    for url in candidates:
        if url in seen:
            continue
        seen.add(url)

        res = run_module(fs, "http_audit", "HTTP audit: {0}".format(url),
                        http_audit.audit, url, scope, timeout=args.timeout,
                        rate=rate, evidence_log=elog)

        if res is None:
            continue
        emit(fs, res, args.brief, not args.no_explain)

    if args.discover:
        def progress(done, total):
            pct = 100.0 * done / max(1, total)
            sys.stdout.write(
                "\r    {0}[*]{0} content discovery {1:.0f}% ({0}/{0})".format(
                    C.GREY, C.RESET, pct, done, total))
            sys.stdout.flush()

        res = run_module(
            fs, "content_discovery",
            "Content discovery: {0}".format(url),
            content_discovery.discover, url, scope,
            wordlist=args.wordlist, delay=args.delay,
            timeout=args.timeout, max_paths=args.max_paths,
            evidence_log=elog, progress=progress,
        )
        sys.stdout.write("\r" + " " * 60 + "\r")
        emit(fs, res, args.brief, not args.no_explain)

# -----
# Knowledge base browsing (no target required)
# -----

def show_kb(term=None):
    base = kb()
    if not base.entries:
        out.error("Knowledge base is empty. Is PyYAML installed, and does "
                 "knowledge/ contain YAML files?")
        return

    if term:
        hits = base.search(term)
        out.section("KNOWLEDGE BASE SEARCH: '{0}'".format(term))
        if not hits:
            out.info("No entries matched.")
            return
    else:
        hits = [(k, base.get(k)) for k in base.all_ids()]
        out.section("KNOWLEDGE BASE - {0} ENTRIES".format(len(hits)))

    for key, entry in hits:
        print()
        print(C.CYAN + "    [{0}] ".format(key) + C.BOLD +
              entry.get("title", "") + C.RESET)
        print(C.GREY + "    " + "-" * 70 + C.RESET)

```

```

        for label, field in [
            ("WHAT", "what"), ("WHY", "why_it_matters"),
            ("HOW", "how_detected"), ("VERIFY", "how_to_verify"),
            ("FIX", "how_to_fix"),
        ]:
            if entry.get(field):
                colour = {"WHY": C.YELLOW, "HOW": C.GREEN,
                           "VERIFY": C.MAGENTA, "FIX": C.BLUE}.get(label, C.WHITE)
                print(colour + " {:<8}".format(label) + C.RESET +
                      out._wrap(entry[field], indent=10))
            if entry.get("concept_tags"):
                print(C.GREY + " {:<8}".format("TAGS") + " " +
                      " . ".join(entry["concept_tags"]) + C.RESET)
        print()

def show_concepts():
    base = kb()
    concepts = base.concepts()
    out.section("CONCEPT INDEX - {}".format(len(concepts)))
    for i in range(0, len(concepts), 3):
        print(" " + " ".join("{:<26}".format(c) for c in concepts[i:i + 3]))
    print()
    out.info("Search any concept with: edrecon.py --kb-search <concept>")
    print()

# -----
# CLI
# -----

def build_parser():
    p = argparse.ArgumentParser(
        prog="edrecon.py",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        description="EDRecon - explainable reconnaissance for teaching.",
        epilog="""
EXAMPLES
Passive only (no packets to target, no scope file needed):
python3 edrecon.py -t example.com --passive-only

Full scan against an authorised lab target:
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml --all

System scan only, custom ports:
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \\\
--system --ports 1-1024

Web scan with content discovery:
python3 edrecon.py -t 192.168.56.101 --scope scope.yaml \\\
--web --discover

Browse the knowledge base with no target at all:
python3 edrecon.py --kb
python3 edrecon.py --kb-search clickjacking
python3 edrecon.py --concepts

Verify an evidence log has not been altered:
python3 edrecon.py --verify-log reports/evidence_xyz.jsonl

AUTHORISED USE ONLY. Active phases require --scope with a valid,
unexpired authorisation file. There is no override.
""",
    )

    p.add_argument("-t", "--target", help="domain, hostname or IP address")
    p.add_argument("-s", "--scope", help="path to authorisation scope file (YAML)")
    p.add_argument("-u", "--url", help="explicit URL for the web phase")

    ph = p.add_argument_group("phases")
    ph.add_argument("--passive-only", action="store_true",
                    help="passive reconnaissance only; no scope file required")
    ph.add_argument("--system", action="store_true", help="active system scan")
    ph.add_argument("--web", action="store_true", help="active web scan")
    ph.add_argument("--all", action="store_true", help="passive + system + web")

    sc = p.add_argument_group("scan options")
    sc.add_argument("--ports", default="top",

```

```

        help="'top' (default), '1-1024', or '22,80,443'")
    sc.add_argument("--threads", type=int, default=50, help="scan threads (default 50)")
    sc.add_argument("--timeout", type=float, default=3.0,
        help="per-connection timeout in seconds (default 3)")
    sc.add_argument("--delay", type=float, default=0.35,
        help="delay between web requests (default 0.35s)")
    sc.add_argument("--no-banners", action="store_true", help="skip banner grabbing")
    sc.add_argument("--axfr", action="store_true",
        help="attempt zone transfer (ACTIVE; needs scope)")
    sc.add_argument("--discover", action="store_true",
        help="run content discovery in the web phase")
    sc.add_argument("--wordlist", help="wordlist file for content discovery")
    sc.add_argument("--max-paths", type=int, default=400,
        help="cap on paths probed (default 400)")
    sc.add_argument("--no-whois", action="store_true", help="skip WHOIS")
    sc.add_argument("--no-ct", action="store_true",
        help="skip certificate transparency lookup")

    o = p.add_argument_group("output")
    o.add_argument("--brief", action="store_true",
        help="one line per finding (suppresses the teaching layer)")
    o.add_argument("--no-explain", action="store_true",
        help="show observations without WHAT/WHY/HOW")
    o.add_argument("--outdir", default="reports", help="report directory")
    o.add_argument("--no-color", action="store_true", help="disable ANSI colour")
    o.add_argument("--no-report", action="store_true", help="skip file output")

    k = p.add_argument_group("knowledge base (no target needed)")
    k.add_argument("--kb", action="store_true", help="print the whole knowledge base")
    k.add_argument("--kb-search", metavar="TERM", help="search the knowledge base")
    k.add_argument("--concepts", action="store_true", help="list all concept tags")
    k.add_argument("--verify-log", metavar="PATH",
        help="verify the hash chain of an evidence log")

    p.add_argument("--version", action="version",
        version="EDRecon {} - EDCatalyst / Dr. Keshav Sinha, UPES".format(VERSION))
    return p

def main():
    parser = build_parser()
    args = parser.parse_args()

    show_banner(no_color=args.no_color)

    # --- knowledge base modes (no target, no scope) -----
    if args.kb:
        show_kb()
        return 0
    if args.kb_search:
        show_kb(args.kb_search)
        return 0
    if args.concepts:
        show_concepts()
        return 0
    if args.verify_log:
        ok, line, msg = EvidenceLog.verify(args.verify_log)
        if ok:
            out.good("Evidence chain intact - no entry has been altered.")
        else:
            out.error("Chain BROKEN at line {}: {}".format(line, msg))
        return 0 if ok else 2

    if not args.target:
        parser.print_help()
        return 1

    target = args.target.strip()
    if "://" in target:
        target = urlparse(target).hostname or target

    do_passive = args.passive_only or args.all or not (args.system or args.web)
    do_system = args.system or args.all
    do_web = args.web or args.all

    # --- scope enforcement -----
    scope = None
    if do_system or do_web or args.axfr:

```

```

try:
    scope = ScopeGuard(args.scope)
except ScopeError as exc:
    out.error(str(exc))
    print()
    out.info("Active scanning is refused without a valid scope file.")
    out.info("Passive reconnaissance needs no scope: "
             "edrecon.py -t {} --passive-only".format(target))
    print()
    return 3

out.section("AUTHORISATION")
print(scope.summary())
print()
try:
    scope.assert_in_scope(target)
    out.good("Target '{}' is within the authorised scope.".format(target))
except OutOfScope as exc:
    out.error(str(exc))
    return 3

# --- run -----
fs = FindingSet(target)
rate = RateLimiter(delay=args.delay)
started = time.time()

stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
safe = "".join(ch if ch.isalnum() or ch in ".-_" else "_" for ch in target)
elog = None
if not args.no_report:
    elog = EvidenceLog(os.path.join(
        args.outdir, "evidence_{}_{}.jsonl".format(safe, stamp)))
    elog.record("scan_start", "edrecon", target,
               {"version": VERSION,
                "phases": {"passive": do_passive, "system": do_system,
                           "web": do_web},
               "scope_file": scope.path if scope else None})

open_ports = []
try:
    if do_passive:
        phase_passive(fs, target, args, rate, elog, scope)
    if do_system:
        open_ports = phase_system(fs, target, args, elog, scope) or []
    if do_web:
        phase_web(fs, target, open_ports, args, rate, elog, scope)
except KeyboardInterrupt:
    print()
    out.warn("Interrupted by user. Reporting what was collected so far.")

elapsed = time.time() - started

# --- summary -----
out.render_summary(fs, elapsed)

if not args.brief and fs.findings:
    out.section("READING THIS REPORT")
    print(C.GREY + "    Confidence levels used above:" + C.RESET)
    for level, meaning in CONFIDENCE_MEANING.items():
        print("    " + C.CYAN + "{:<11}".format(level) + C.RESET +
              out._wrap(meaning, indent=14))
    print()

if not args.no_report:
    jpath = os.path.join(args.outdir, "report_{}_{}.json".format(safe, stamp))
    hpath = os.path.join(args.outdir, "report_{}_{}.html".format(safe, stamp))
    out.save_json(fs, jpath)
    out.save_html(fs, hpath)
    if elog:
        elog.record("scan_end", "edrecon", target,
                   {"findings": len(fs), "seconds": round(elapsed, 1)})

    out.section("ARTEFACTS")
    out.good("HTML report : " + hpath)
    out.good("JSON report : " + jpath)
    if elog:
        out.good("Evidence log: " + elog.path)
        print(C.GREY + "                                verify with: python3 edrecon.py "

```

```
        "--verify-log " + elog.path + C.RESET)
    print()
    return 0

if __name__ == "__main__":
    try:
        sys.exit(main())
    except KeyboardInterrupt:
        print("\n[!] Aborted.")
        sys.exit(130)
```

core/finding.py

The Finding dataclass - the teaching layer schema. Every module returns these, never print statements.

```

"""
The Finding object.

This is the heart of EDRecon. Every module returns Findings, never raw
print statements. A Finding carries not only WHAT was observed but WHY it
matters and HOW it was detected -- so the output teaches, rather than
merely reporting.
"""

from dataclasses import dataclass, field, asdict
from datetime import datetime, timezone
from typing import Any, Dict, List

# -----
# Severity and confidence vocabularies
# -----

SEVERITY_ORDER = ["info", "low", "medium", "high", "critical"]

SEVERITY_MEANING = {
    "info": "Neutral fact about the target. Not a weakness by itself.",
    "low": "Minor hygiene issue. Useful to an attacker only in combination.",
    "medium": "Real weakness. Meaningfully expands attack surface.",
    "high": "Serious weakness. Direct path toward compromise.",
    "critical": "Severe. Compromise likely trivial from here.",
}

CONFIDENCE_MEANING = {
    "confirmed": (
        "Directly proven by the evidence. The target itself told us this, "
        "or we completed a full protocol exchange."
    ),
    "probable": (
        "Strongly indicated but not proven. Based on a distinctive signature "
        "that is rarely wrong."
    ),
    "inferred": (
        "A best guess from indirect signals. Could be defeated by "
        "misconfiguration, a proxy, or deliberate deception."
    ),
}

@dataclass
class Finding:
    """A single explainable observation about a target."""

    # --- identity -----
    id: str = field() # stable ID, e.g. "TLS-003"
    module: str = field() # module that produced it
    target: str = field() # host / URL the finding concerns
    title: str = field() # one-line human summary

    # --- the raw observation -----
    observation: str = field() # WHAT was seen, factually
    evidence: Dict[str, Any] = field(default_factory=dict)

    # --- the teaching layer -----
    what: str = field() # what this technically means
    why_it_matters: str = field() # security significance / attacker use
    how_detected: str = field() # the mechanism that revealed it
    how_to_verify: str = field() # manual command to reproduce
    how_to_fix: str = field() # defender-side remediation

    # --- classification -----
    # Left empty by default ON PURPOSE. An empty severity lets enrich()
    # fill it from the knowledge base; a default of "info" would be truthy
    # and silently suppress every KB severity.
    severity: str = field()
    confidence: str = field()
    references: List[str] = field(default_factory=list)
    concept_tags: List[str] = field(default_factory=list)

    # --- provenance -----
    timestamp: str = field(
        default_factory=lambda: datetime.now(timezone.utc).isoformat()
    )
    passive: bool = True # True if no packet was sent to target

```

```

def __post_init__(self):
    # "" is permitted here -- enrich() resolves it. Anything else
    # invalid is coerced.
    if self.severity and self.severity not in SEVERITY_ORDER:
        self.severity = "info"
    if self.confidence not in CONFIDENCE_MEANING:
        self.confidence = "inferred"

@property
def severity_rank(self) -> int:
    if self.severity not in SEVERITY_ORDER:
        return 0
    return SEVERITY_ORDER.index(self.severity)

def to_dict(self) -> Dict[str, Any]:
    return asdict(self)

def enrich(self, kb_entry: Dict[str, Any]) -> "Finding":
    """
    Fill the teaching fields from a knowledge-base entry.

    Values already set on the Finding win, so a module can override
    the generic explanation with a target-specific one.
    """
    if not kb_entry:
        return self
    for key in (
        "title",
        "what",
        "why_it_matters",
        "how_detected",
        "how_to_verify",
        "how_to_fix",
        "severity",
    ):
        if not getattr(self, key, "") and kb_entry.get(key):
            setattr(self, key, kb_entry[key])
    for key in ("references", "concept_tags"):
        if not getattr(self, key) and kb_entry.get(key):
            setattr(self, key, list(kb_entry[key]))

    # Whatever happened above, end with a valid severity.
    if self.severity not in SEVERITY_ORDER:
        self.severity = "info"
    return self

class FindingSet:
    """A collection of findings for one scan run."""

    def __init__(self, target: str):
        self.target = target
        self.findings: List[Finding] = []
        self.started = datetime.now(timezone.utc)
        self.errors: List[Dict[str, str]] = []

    def add(self, finding: Finding):
        self.findings.append(finding)

    def extend(self, findings):
        for f in findings or []:
            self.add(f)

    def record_error(self, module: str, message: str):
        """A module failing must never stop the run."""
        self.errors.append({"module": module, "error": str(message)})

    def sorted_by_severity(self) -> List[Finding]:
        return sorted(
            self.findings, key=lambda f: (-f.severity_rank, f.module, f.id)
        )

    def by_severity(self) -> Dict[str, int]:
        counts = {s: 0 for s in SEVERITY_ORDER}
        for f in self.findings:
            counts[f.severity] += 1
        return counts

    def concepts_touched(self) -> List[str]:
        """Used to build the 'what you learned' summary."""
        tags = set()
        for f in self.findings:
            tags.update(f.concept_tags)
        return sorted(tags)

    def __len__(self):
        return len(self.findings)

```

core/scope_guard.py

Authorisation enforcement. Loads the scope file and gates every active module. No bypass exists.

```

"""
Scope guard.

Every ACTIVE module must call ScopeGuard.assert_in_scope() before it sends
a single packet. There is deliberately no --force flag and no bypass.

The scope file IS the authorisation record. In a real engagement it mirrors
the signed rules-of-engagement; in the classroom it mirrors the lab sheet.
Teaching students to treat it as mandatory is the point.
"""

import ipaddress
import os
import socket
from datetime import date, datetime
from typing import List, Optional

try:
    import yaml
except ImportError: # pragma: no cover
    yaml = None

class ScopeError(Exception):
    """Raised when a target is not authorised. Never caught silently."""

class OutOfScope(ScopeError):
    pass

class ScopeGuard:
    """Loads and enforces a scope file."""

    def __init__(self, scope_path: str):
        if yaml is None:
            raise ScopeError(
                "PyYAML is required for scope enforcement. "
                "Install it with: pip install pyyaml"
            )
        if not scope_path:
            raise ScopeError(
                "No scope file supplied. Active modules cannot run without "
                "an authorisation scope. Use --scope <file>."
            )
        if not os.path.isfile(scope_path):
            raise ScopeError("Scope file not found: {}".format(scope_path))

        self.path = os.path.abspath(scope_path)
        with open(scope_path, "r", encoding="utf-8") as fh:
            self.data = yaml.safe_load(fh) or {}

        self._validate()

        self.engagement = self.data.get("engagement", {})
        self.hosts: List[str] = [
            str(h) for h in (self.data.get("in_scope", {}).get("hosts") or [])
        ]
        self.networks: List[str] = [
            str(n) for n in (self.data.get("in_scope", {}).get("networks") or [])
        ]
        self.domains: List[str] = [
            str(d).lower().lstrip(".")
            for d in (self.data.get("in_scope", {}).get("domains") or [])
        ]
        self.excluded: List[str] = [
            str(x) for x in (self.data.get("out_of_scope") or [])
        ]

        self._networks = []
        for net in self.networks:
            try:
                self._networks.append(ipaddress.ip_network(net, strict=False))
            except ValueError:
                raise ScopeError("Invalid network in scope file: {}".format(net))

        self._excluded_nets = []
        self._excluded_hosts = []

```

```

    for item in self.excluded:
        try:
            self._excluded_nets.append(ipaddress.ip_network(item, strict=False))
        except ValueError:
            self._excluded_hosts.append(item.lower())

# -- validation -----
def _validate(self):
    eng = self.data.get("engagement")
    if not isinstance(eng, dict):
        raise ScopeError(
            "Scope file must contain an 'engagement:' block with "
            "authorised_by, contact and expires fields."
        )

    for required in ("name", "authorised_by", "contact", "expires"):
        if not eng.get(required):
            raise ScopeError(
                "Scope file is missing required field "
                "'engagement.{}'.format(required)
            )

    expires = eng.get("expires")
    if isinstance(expires, str):
        try:
            expires = datetime.strptime(expires, "%Y-%m-%d").date()
        except ValueError:
            raise ScopeError(
                "engagement.expires must be YYYY-MM-DD, got: "
                "{}".format(expires)
            )
    if isinstance(expires, datetime):
        expires = expires.date()
    if not isinstance(expires, date):
        raise ScopeError("engagement.expires must be a date (YYYY-MM-DD).")

    if expires < date.today():
        raise ScopeError(
            "AUTHORISATION EXPIRED on {}. This scope is no longer valid. "
            "Obtain fresh written authorisation before scanning.".format(
                expires.isoformat()
            )
        )
    self.expires = expires

    scope = self.data.get("in_scope") or {}
    if not any(scope.get(k) for k in ("hosts", "networks", "domains")):
        raise ScopeError(
            "Scope file defines no in-scope hosts, networks or domains."
        )

# -- resolution -----
@staticmethod
def _resolve(target: str) -> Optional[str]:
    try:
        return socket.gethostbyname(target)
    except (socket.gaierror, UnicodeError, OSError):
        return None

@staticmethod
def _strip(target: str) -> str:
    t = target.strip()
    for prefix in ("https://", "http://"):
        if t.lower().startswith(prefix):
            t = t[len(prefix):]
    t = t.split("/")[0]
    if ":" in t and not t.count(":") > 1: # strip port, keep IPv6
        t = t.split(":")[0]
    return t.lower().rstrip(".")

# -- the check -----
def is_in_scope(self, target: str) -> bool:
    host = self._strip(target)
    if not host:
        return False

```

```

# Explicit exclusions always win.
if host in self._excluded_hosts:
    return False

ip = None
try:
    ip = ipaddress.ip_address(host)
except ValueError:
    resolved = self._resolve(host)
    if resolved:
        try:
            ip = ipaddress.ip_address(resolved)
        except ValueError:
            ip = None

if ip is not None:
    for net in self._excluded_nets:
        if ip in net:
            return False

# Literal host match
if host in [h.lower() for h in self.hosts]:
    return True

# Domain / subdomain match
for dom in self.domains:
    if host == dom or host.endswith("." + dom):
        return True

# Network membership
if ip is not None:
    for net in self._networks:
        if ip in net:
            return True
    if str(ip) in self.hosts:
        return True

return False

def assert_in_scope(self, target: str):
    """
    Called by every active module before sending traffic.
    Raises OutOfScope -- which the CLI does NOT swallow.
    """
    if not self.is_in_scope(target):
        raise OutOfScope(
            "TARGET NOT AUTHORISED: '{}' is not covered by the scope "
            "file '{}'.\n"
            "Active scanning refused. Add the target to the scope "
            "file only if you hold written authorisation for it.".format(
                target, os.path.basename(self.path)
            )
        )

# -- display -----

def summary(self) -> str:
    lines = [
        "Engagement : {}".format(self.engagement.get("name")),
        "Authorised : {}".format(self.engagement.get("authorised_by")),
        "Contact : {}".format(self.engagement.get("contact")),
        "Expires : {}".format(self.expires.isoformat()),
    ]
    if self.hosts:
        lines.append("Hosts : {}".format(", ".join(self.hosts)))
    if self.networks:
        lines.append("Networks : {}".format(", ".join(self.networks)))
    if self.domains:
        lines.append("Domains : {}".format(", ".join(self.domains)))
    if self.excluded:
        lines.append("EXCLUDED : {}".format(", ".join(self.excluded)))
    return "\n".join(" " + ln for ln in lines)

```

core/knowledge.py

Loads the YAML knowledge base that supplies all explanatory text.

```
"""
Knowledge base loader.

Explanations live in knowledge/*.yaml, NOT in module code. This means the
pedagogy can be edited, translated and version-controlled independently of
the scanner logic -- and students can read the KB as a reference even with
no target at all.
"""

import os
import glob

try:
    import yaml
except ImportError:
    yaml = None

KB_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "knowledge")

class KnowledgeBase:
    def __init__(self, kb_dir=None):
        self.kb_dir = kb_dir or KB_DIR
        self.entries = {}
        self._load()

    def _load(self):
        if yaml is None:
            return
        for path in sorted(glob.glob(os.path.join(self.kb_dir, "*.yaml"))):
            try:
                with open(path, "r", encoding="utf-8") as fh:
                    data = yaml.safe_load(fh) or {}
                    for key, val in data.items():
                        if isinstance(val, dict):
                            self.entries[key] = val
            except Exception:
                continue

    def get(self, finding_id):
        return self.entries.get(finding_id, {})

    def all_ids(self):
        return sorted(self.entries.keys())

    def search(self, term):
        term = term.lower()
        hits = []
        for key, val in self.entries.items():
            blob = " ".join(str(v) for v in val.values()).lower()
            if term in key.lower() or term in blob:
                hits.append((key, val))
        return hits

    def concepts(self):
        tags = set()
        for val in self.entries.values():
            tags.update(val.get("concept_tags", []) or [])
        return sorted(tags)

_KB = None

def kb():
    global _KB
    if _KB is None:
        _KB = KnowledgeBase()
    return _KB
```

core/evidence.py

Rate limiting and the hash-chained evidence log.

```
"""
Rate limiting and evidence logging.

Two things most teaching tools get wrong:

1. They hammer third-party services with no throttle and no User-Agent,
   then students learn nothing except how to get HTTP 403'd.
2. They print results to a terminal that scrolls away, so there is no
   artefact to submit, grade, or reason about later.

RateLimiter fixes the first. EvidenceLog fixes the second, and doubles as
a chain-of-custody demonstration for the forensics side of the syllabus.
"""

import hashlib
import json
import os
import threading
import time
from datetime import datetime, timezone

USER_AGENT = (
    "EDRecon/1.0 (EDCatalyst teaching tool; UPES Dehradun; "
    "+https://edcatalyst.in)"
)

class RateLimiter:
    """Simple per-host throttle. Honours Retry-After when told to."""

    def __init__(self, delay=1.0):
        self.delay = float(delay)
        self._last = {}
        self._lock = threading.Lock()

    def wait(self, host):
        with self._lock:
            now = time.monotonic()
            last = self._last.get(host, 0.0)
            gap = now - last
            if gap < self.delay:
                time.sleep(self.delay - gap)
            self._last[host] = time.monotonic()

    def backoff(self, host, seconds):
        try:
            seconds = float(seconds)
        except (TypeError, ValueError):
            seconds = self.delay * 5
        seconds = min(seconds, 60.0)
        time.sleep(seconds)
        with self._lock:
            self._last[host] = time.monotonic()

class EvidenceLog:
    """
    Append-only JSONL log. Every entry is hashed, and each hash includes
    the previous hash -- so tampering with an earlier line invalidates
    every line after it. That is the same principle as a forensic hash
    chain, at a scale students can inspect by hand.
    """

    def __init__(self, path):
        self.path = path
        self._prev_hash = "0" * 64
        self._lock = threading.Lock()
        os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
        with open(self.path, "w", encoding="utf-8") as fh:
            fh.write("")

    def record(self, event_type, module, target, payload=None):
        entry = {
            "ts": datetime.now(timezone.utc).isoformat(),
            "event": event_type,
            "module": module,
            "target": target,
            "payload": payload or {},
        }
```

```
        "prev_hash": self._prev_hash,
    }
    body = json.dumps(entry, sort_keys=True, ensure_ascii=False)
    digest = hashlib.sha256(body.encode("utf-8")).hexdigest()
    entry["hash"] = digest

    with self._lock:
        with open(self.path, "a", encoding="utf-8") as fh:
            fh.write(json.dumps(entry, ensure_ascii=False) + "\n")
        self._prev_hash = digest
    return digest

@staticmethod
def verify(path):
    """Re-walk the chain and report the first broken link, if any."""
    prev = "0" * 64
    with open(path, "r", encoding="utf-8") as fh:
        for lineno, line in enumerate(fh, 1):
            line = line.strip()
            if not line:
                continue
            entry = json.loads(line)
            claimed = entry.pop("hash", None)
            if entry.get("prev_hash") != prev:
                return False, lineno, "prev_hash mismatch"
            body = json.dumps(entry, sort_keys=True, ensure_ascii=False)
            actual = hashlib.sha256(body.encode("utf-8")).hexdigest()
            if actual != claimed:
                return False, lineno, "content hash mismatch"
            prev = claimed
    return True, None, "chain intact"
```

core/output.py

Console and HTML renderers.

```
"""
Renderers.

Console output is the teaching surface: each finding is printed as a
labelled block (OBSERVED / WHAT / WHY / HOW / VERIFY / FIX) rather than a
single cryptic line. --brief collapses this to one line per finding for
users who already know the material.
"""

import html
import json
import os
import textwrap
from datetime import datetime, timezone

from core.banner import C, VERSION
from core.finding import SEVERITY_MEANING, CONFIDENCE_MEANING

SEV_COLOR = {
    "info": C.BLUE,
    "low": C.CYAN,
    "medium": C.YELLOW,
    "high": C.RED,
    "critical": C.MAGENTA,
}

SEV_HTML = {
    "info": "#3b82f6",
    "low": "#06b6d4",
    "medium": "#f59e0b",
    "high": "#ef4444",
    "critical": "#a855f7",
}

WIDTH = 74

def _wrap(text, indent=11):
    if not text:
        return ""
    pad = " " * indent
    return textwrap.fill(
        str(text).strip(),
        width=WIDTH + indent,
        initial_indent="",
        subsequent_indent=pad,
    )

def info(msg):
    print(C.BLUE + "[i] " + C.RESET + msg)

def good(msg):
    print(C.GREEN + "[+] " + C.RESET + msg)

def warn(msg):
    print(C.YELLOW + "[!] " + C.RESET + msg)

def error(msg):
    print(C.RED + "[x] " + C.RESET + msg)

def step(msg):
    print(C.CYAN + "\n[>] " + C.BOLD + msg + C.RESET)

def section(title):
    print()
    print(C.CYAN + "=" * WIDTH + C.RESET)
    print(C.BOLD + C.WHITE + " " + title + C.RESET)
    print(C.CYAN + "=" * WIDTH + C.RESET)

def render_finding(f, brief=False, explain=True):
    """Print a single finding."""
```

```

color = SEV_COLOR.get(f.severity, C.WHITE)

if brief:
    print(
        "{}[{:<8}]{} {:<10} {} {}".format(
            color,
            f.severity.upper(),
            C.RESET,
            f.id,
            f.title,
            C.GREY + "(" + f.target + ")" + C.RESET,
        )
    )
    return

print()
print(
    color
    + " ["
    + f.id
    + "]"
    + C.BOLD
    + f.title
    + C.RESET
    + C.GREY
    + " "
    + "." * max(2, WIDTH - len(f.id) - len(f.title) - 20)
    + " "
    + f.severity
    + C.RESET
)
print(C.GREY + " " + "-" * (WIDTH - 2) + C.RESET)

mode = "PASSIVE" if f.passive else "ACTIVE"
print(
    C.GREY
    + " {:<9}".format("SOURCE")
    + C.RESET
    + "{} module '{}' | confidence: {}".format(mode, f.module, f.confidence)
)

if f.observation:
    print(C.WHITE + " {:<9}".format("OBSERVED") + C.RESET + _wrap(f.observation))
if explain:
    if f.what:
        print(C.CYAN + " {:<9}".format("WHAT") + C.RESET + _wrap(f.what))
    if f.why_it_matters:
        print(C.YELLOW + " {:<9}".format("WHY") + C.RESET + _wrap(f.why_it_matters))
    if f.how_detected:
        print(C.GREEN + " {:<9}".format("HOW") + C.RESET + _wrap(f.how_detected))
    if f.how_to_verify:
        print(C.MAGENTA + " {:<9}".format("VERIFY") + C.RESET + _wrap(f.how_to_verify))
    if f.how_to_fix:
        print(C.BLUE + " {:<9}".format("FIX") + C.RESET + _wrap(f.how_to_fix))
    if f.references:
        print(C.GREY + " {:<9}".format("REFS") + " " + " | ".join(f.references) + C.RESET)
    if f.concept_tags:
        print(C.GREY + " {:<9}".format("TAGS") + " " + " . ".join(f.concept_tags) + C.RESET)

def render_summary(fs, elapsed):
    """End-of-scan summary, including the 'what you learned' block."""
    section("SCAN SUMMARY")

    counts = fs.by_severity()
    print(" Target      : " + C.BOLD + fs.target + C.RESET)
    print(" Findings     : " + str(len(fs)))
    print(" Duration     : {:.1f}s".format(elapsed))
    print()
    for sev in ["critical", "high", "medium", "low", "info"]:
        if counts.get(sev):
            print(
                " {}{:<10}{} {:>3} {}".format(
                    SEV_COLOR.get(sev, ""),
                    sev,
                    C.RESET,
                    counts[sev],
                    C.GREY + SEVERITY_MEANING[sev],
                )
            )

```

```

        C.RESET,
    )
)

if fs.errors:
    print()
    warn("{} module(s) reported errors (scan continued):".format(len(fs.errors)))
    for e in fs.errors:
        print(C.GREY + "          {}: {}".format(e["module"], e["error"]) + C.RESET)

concepts = fs.concepts_touched()
if concepts:
    section("WHAT THIS SCAN TAUGHT YOU")
    print(
        C.GREY
        + "  Concepts exercised in this run -- revisit any that are unfamiliar:"
        + C.RESET
    )
    print()
    for i in range(0, len(concepts), 3):
        row = concepts[i:i + 3]
        print("    " + "".join(C.CYAN + "*" + C.RESET + "{:<22}".format(c) for c in row))
print()

def save_json(fs, path):
    data = {
        "tool": "EDRecon",
        "version": VERSION,
        "author": "Dr. Keshav Sinha, UPES, India",
        "project": "EDCatalyst",
        "target": fs.target,
        "generated": datetime.now(timezone.utc).isoformat(),
        "summary": fs.by_severity(),
        "concepts": fs.concepts_touched(),
        "errors": fs.errors,
        "findings": [f.to_dict() for f in fs.sorted_by_severity()],
    }
    os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
    with open(path, "w", encoding="utf-8") as fh:
        json.dump(data, fh, indent=2, ensure_ascii=False)
    return path

def save_html(fs, path):
    e = html.escape
    counts = fs.by_severity()

    cards = []
    for f in fs.sorted_by_severity():
        color = SEV_HTML.get(f.severity, "#666")
        rows = []
        for label, value, cls in [
            ("OBSERVED", f.observation, "obs"),
            ("WHAT", f.what, "what"),
            ("WHY IT MATTERS", f.why_it_matters, "why"),
            ("HOW DETECTED", f.how_detected, "how"),
            ("HOW TO VERIFY", f.how_to_verify, "verify"),
            ("HOW TO FIX", f.how_to_fix, "fix"),
        ]:
            if value:
                rows.append(
                    '<div class="row {}"><span class="lbl">{}</span>'
                    '<span class="val">{}</span></div>'.format(cls, label, e(str(value)))
                )
        refs = ""
        if f.references:
            refs = '<div class="refs">' + " &middot; ".join(e(r) for r in f.references) + "</div>"
        tags = ""
        if f.concept_tags:
            tags = '<div class="tags">' + "".join(
                '<span class="tag">{}</span>'.format(e(t)) for t in f.concept_tags
            ) + "</div>"
        ev = ""
        if f.evidence:
            ev = '<details><summary>Raw evidence</summary><pre>{}</pre></details>'.format(
                e(json.dumps(f.evidence, indent=2, default=str))
            )

```

```

cards.append(
    """
<div class="card" style="border-left-color:{color}">
  <div class="head">
    <span class="fid">{fid}</span>
    <span class="title">{title}</span>
    <span class="sev" style="background:{color}">{sev}</span>
  </div>
  <div class="meta">{mode} &middot; module <code>{module}</code>
    &middot; confidence <b>{conf}</b> &middot; {target}</div>
  {rows}{refs}{tags}{ev}
</div>""".format(
    color=color,
    fid=e(f.id),
    title=e(f.title),
    sev=e(f.severity.upper()),
    mode="PASSIVE" if f.passive else "ACTIVE",
    module=e(f.module),
    conf=e(f.confidence),
    target=e(f.target),
    rows="" .join(rows),
    refs=refs,
    tags=tags,
    ev=ev,
)
)

chips = "" .join(
    '<span class="chip" style="background:{color}">{color}</span>'.format(
        SEV_HTML[s], counts[s], s
    )
    for s in ["critical", "high", "medium", "low", "info"]
    if counts.get(s)
)

concepts = "" .join(
    '<span class="tag">{tag}</span>'.format(e(c)) for c in fs.concepts_touched()
)

doc = """<!DOCTYPE html>
<html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>EDRecon Report - {target}</title>
<style>
:root {{ --bg:#0f172a; --panel:#1e293b; --ink:#e2e8f0; --dim:#94a3b8; }}
* {{ box-sizing:border-box; }}
body {{ margin:0; background:var(--bg); color:var(--ink);
font-family:ui-sans-serif,system-ui,'Segoe UI',Roboto,sans-serif;
line-height:1.6; }}
header {{ background:linear-gradient(135deg,#0891b2,#0e7490); padding:28px 32px; }}
header h1 {{ margin:0; font-size:26px; letter-spacing:1px; }}
header .sub {{ opacity:.9; font-size:13px; margin-top:4px; }}
header .auth {{ margin-top:14px; font-size:12px; opacity:.85;
border-top:1px solid rgba(255,255,255,.25); padding-top:10px; }}
.wrap {{ max-width:960px; margin:0 auto; padding:24px 32px 60px; }}
.bar {{ display:flex; gap:8px; flex-wrap:wrap; margin:18px 0 26px; }}
.chip {{ padding:5px 13px; border-radius:20px; font-size:12px;
font-weight:600; color:#0f172a; }}
.card {{ background:var(--panel); border-left:5px solid #666; border-radius:8px;
padding:18px 20px; margin-bottom:18px; }}
.head {{ display:flex; align-items:center; gap:10px; flex-wrap:wrap; }}
.fid {{ font-family:ui-monospace,monospace; font-size:12px; color:var(--dim); }}
.title {{ font-weight:600; font-size:16px; flex:1; }}
.sev {{ font-size:10px; font-weight:700; color:#0f172a;
padding:3px 9px; border-radius:4px; letter-spacing:.5px; }}
.meta {{ font-size:11.5px; color:var(--dim); margin:6px 0 14px; }}
.row {{ display:flex; gap:14px; margin-bottom:9px; align-items:flex-start; }}
.lbl {{ flex:0 0 118px; font-size:10.5px; font-weight:700; letter-spacing:.6px;
color:var(--dim); padding-top:3px; }}
.val {{ flex:1; font-size:14px; }}
.why .val {{ color:#fde68a; }}
.how .val {{ color:#bbf7d0; }}
.verify .val {{ font-family:ui-monospace,monospace; font-size:12.5px; color:#e9d5ff; }}
.fix .val {{ color:#bdfbfe; }}
.refs {{ font-size:11px; color:var(--dim); margin-top:12px;
border-top:1px solid #334155; padding-top:9px; }}
.tags {{ margin-top:9px; }}

```

```

.tag {{ display:inline-block; background:#334155; color:#cbd5e1; font-size:11px;
padding:2px 9px; border-radius:4px; margin:2px 5px 2px 0; }}
details {{ margin-top:11px; }}
summary {{ cursor:pointer; font-size:12px; color:var(--dim); }}
pre {{ background:#0b1220; padding:12px; border-radius:6px; overflow-x:auto;
font-size:11.5px; color:#94a3b8; }}
h2 {{ font-size:15px; letter-spacing:1px; color:var(--dim);
border-bottom:1px solid #334155; padding-bottom:8px; margin-top:36px; }}
footer {{ text-align:center; color:var(--dim); font-size:11.5px;
padding:26px; border-top:1px solid #334155; }}
</style></head><body>
<header>
<h1>EDRecon Reconnaissance Report</h1>
<div class="sub">Target: <b>{target}</b> &middot; Generated {when}</div>
<div class="auth">EDCatalyst &middot; Dr. Keshav Sinha &middot; UPES, Dehradun, India
&middot; edcatalyst.in</div>
</header>
<div class="wrap">
<div class="bar">{chips}</div>
<h2>FINDINGS</h2>
{cards}
<h2>CONCEPTS EXERCISED</h2>
<div class="tags">{concepts}</div>
</div>
<footer>
Generated by EDRecon v{version} &middot; Authorised testing only.<br>
This report is a teaching artefact. Verify every finding independently.
</footer>
</body></html>""".format(
    target=e(fs.target),
    when=datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC"),
    chips=chips,
    cards="".join(cards) or "<p>No findings recorded.</p>",
    concepts=concepts or "<i>none</i>",
    version=VERSION,
)

os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
with open(path, "w", encoding="utf-8") as fh:
    fh.write(doc)
return path

```



```
""" + C.RESET)
print(C.CYAN + line + C.RESET)
print(
    C.WHITE
    + "    EDRecon "
    + C.GREEN
    + "v"
    + VERSION
    + C.WHITE
    + " | Explainable Reconnaissance Framework"
    + C.RESET
)
print(
    C.GREY
    + "    An EDCatalyst teaching tool - findings that explain WHAT, WHY and HOW"
    + C.RESET
)
print(C.CYAN + "-" * 74 + C.RESET)
print(C.WHITE + "    Author : " + C.YELLOW + "Dr. Keshav Sinha" + C.RESET)
print(
    C.WHITE
    + "    Affil. : "
    + C.YELLOW
    + "UPES, Dehradun, India"
    + C.RESET
)
print(C.WHITE + "    Project : " + C.YELLOW + "EDCatalyst (edcatalyst.in)" + C.RESET)
print(C.CYAN + line + C.RESET)
print(
    C.RED
    + "    [!] AUTHORISED USE ONLY. "
    + C.WHITE
    + "Active modules require a signed scope file."
    + C.RESET
)
print(
    C.GREY
    + "    Unauthorised scanning may violate the IT Act 2000 (India) ss.43/66"
    + C.RESET
)
print(C.CYAN + line + C.RESET)
print()
```

modules/passive/dns_recon.py

PASSIVE - DNS, SPF, DMARC, and the AXFR check.

```
"""
PASSIVE - DNS reconnaissance.

No packet reaches the target. Queries go to a recursive resolver, which
does the work on our behalf. This is the cleanest illustration of what
"passive" actually means, and worth stressing in class: passive does not
mean undetectable, it means the target's own logs show nothing.

AXFR is the one exception and is labelled ACTIVE, because a zone transfer
request goes directly to the target's authoritative nameserver.
"""

from core.finding import Finding
from core.knowledge import kb

MODULE = "dns_recon"

try:
    import dns.resolver
    import dns.query
    import dns.zone
    import dns.exception
    HAVE_DNS = True
except ImportError:
    HAVE_DNS = False

def _mk(fid, target, observation, evidence=None, **kw):
    f = Finding(id=fid, module=MODULE, target=target,
               title="", observation=observation,
               evidence=evidence or {}, **kw)
    return f.enrich(kb().get(fid))

def run(target, scope=None, evidence_log=None, opts=None):
    """Returns a list of Findings. Never raises."""
    opts = opts or {}
    findings = []

    if not HAVE_DNS:
        raise RuntimeError(
            "dnspython not installed. Run: pip install dnspython"
        )

    resolver = dns.resolver.Resolver()
    resolver.timeout = float(opts.get("timeout", 5))
    resolver.lifetime = float(opts.get("timeout", 5))

    def q(name, rtype):
        try:
            return resolver.resolve(name, rtype)
        except Exception:
            return []

    # --- A / AAAA -----
    addrs = []
    for rtype in ("A", "AAAA"):
        for rdata in q(target, rtype):
            addrs.append(str(rdata))
    if addrs:
        findings.append(_mk(
            "DNS-001", target,
            "Resolved to: {}".format(", ".join(addrs)),
            {"addresses": addrs},
            passive=True, confidence="confirmed",
        ))
    if evidence_log:
        evidence_log.record("dns_resolve", MODULE, target, {"addresses": addrs})

    # --- NS -----
    nameservers = [str(r).rstrip(".") for r in q(target, "NS")]
    if nameservers:
        findings.append(_mk(
            "DNS-007", target,
            "Authoritative nameservers: {}".format(", ".join(nameservers)),
            {"nameservers": nameservers},
            passive=True,
        ))
```

```

# --- MX -----
mx = []
for r in q(target, "MX"):
    mx.append(str(r))
if mx:
    provider = "self-hosted or unknown"
    blob = " ".join(mx).lower()
    if "google" in blob or "googlemail" in blob:
        provider = "Google Workspace"
    elif "outlook" in blob or "protection.outlook" in blob:
        provider = "Microsoft 365"
    elif "zoho" in blob:
        provider = "Zoho Mail"
    findings.append(_mk(
        "DNS-002", target,
        "MX records: {} (provider appears to be {})".format(
            " ".join(mx), provider),
        {"mx": mx, "provider_guess": provider},
        passive=True,
        confidence="probable" if provider != "self-hosted or unknown" else "confirmed",
    ))

# --- TXT / SPF -----
txts = []
for r in q(target, "TXT"):
    txt = str(r).strip("'").replace(' ', '')
    txts.append(txt)

spf = [t for t in txts if t.lower().startswith("v=spf1")]
if spf:
    record = spf[0]
    includes = [
        part.split(":", 1)[1]
        for part in record.split()
        if part.lower().startswith("include:")
    ]
    mech = "-all" if "-all" in record else ("~all" if "~all" in record else "unspecified")
    findings.append(_mk(
        "DNS-003", target,
        "SPF present, terminates with '{}'. Third-party senders: {}".format(
            mech, " ".join(includes) if includes else "none"),
        {"record": record, "includes": includes, "all_mechanism": mech},
        passive=True,
    ))
else:
    findings.append(_mk(
        "DNS-004", target,
        "No v=spf1 TXT record published for this domain.",
        {"txt_records_seen": len(txts)},
        passive=True,
    ))

other_txt = [
    t for t in txts
    if not t.lower().startswith("v=spf1") and not t.lower().startswith("v=dmARC1")
]
if other_txt:
    findings.append(_mk(
        "DNS-009", target,
        "{} non-SPF TXT record(s) present, typically SaaS "
        "verification tokens.".format(len(other_txt)),
        {"records": other_txt[:20]},
        passive=True,
    ))

# --- DMARC -----
dmarc = []
for r in q("_dmarc." + target, "TXT"):
    dmarc.append(str(r).strip("'").replace(' ', ''))
dmarc = [d for d in dmarc if d.lower().startswith("v=dmARC1")]
if dmarc:
    record = dmarc[0]
    policy = "none"
    for part in record.split(";"):
        part = part.strip()
        if part.lower().startswith("p="):
            policy = part.split("=", 1)[1].strip()

```

```

sev = "medium" if policy.lower() == "none" else "info"
extra = ""
if policy.lower() == "none":
    extra = (
        " Policy is p=none, which only monitors - spoofed mail is "
        "still delivered."
    )
f = _mk(
    "DNS-005", target,
    "DMARC published with policy p={}.{}".format(policy, extra),
    {"record": record, "policy": policy},
    passive=True,
)
f.severity = sev
findings.append(f)
else:
    findings.append(_mk(
        "DNS-006", target,
        "No DMARC record at _dmarc.{}".format(target),
        {}, passive=True,
    ))

# --- AXFR (ACTIVE - scope gated) -----
if opts.get("try_axfr") and nameservers:
    if scope is None:
        raise RuntimeError("AXFR requires a scope file (active check).")
    scope.assert_in_scope(target)
    for ns in nameservers:
        try:
            ns_ip = str(resolver.resolve(ns, "A")[0])
        except Exception:
            continue
        try:
            zone = dns.zone.from_xfr(
                dns.query.xfr(ns_ip, target, timeout=8, lifetime=8)
            )
            names = [str(n) for n in zone.nodes.keys()]
            findings.append(_mk(
                "DNS-008", target,
                "Nameserver {} returned the full zone: {} records "
                "disclosed.".format(ns, len(names)),
                {"nameserver": ns, "record_count": len(names),
                 "sample": names[:40]},
                passive=False, confidence="confirmed",
            ))
            if evidence_log:
                evidence_log.record("axfr_success", MODULE, target,
                                    {"nameserver": ns, "records": len(names)})
        except Exception:
            continue # refused = correctly configured

    return findings

def enumerate_subdomains(target, names, opts=None):
    """Resolve a candidate list, return Findings for those that exist."""
    findings = []
    if not HAVE_DNS:
        return findings
    resolver = dns.resolver.Resolver()
    resolver.timeout = 3
    resolver.lifetime = 3
    seen = set()
    for name in names:
        if name in seen:
            continue
        seen.add(name)
        try:
            answers = resolver.resolve(name, "A")
            ips = [str(a) for a in answers]
            findings.append(_mk(
                "DNS-010", name,
                "Subdomain resolves to {}".format(", ".join(ips)),
                {"host": name, "addresses": ips},
                passive=True,
            ))
        except Exception:
            continue

    return findings

```

modules/passive/whois_ct.py

PASSIVE - WHOIS registration and certificate transparency subdomain harvesting.

```

"""
PASSIVE - WHOIS registration data and Certificate Transparency logs.

Certificate Transparency deserves emphasis in teaching. Every publicly
trusted certificate issued since 2018 is written to a public append-only
log. Querying crt.sh therefore enumerates subdomains without sending a
single packet to the target - the organisation published this data itself,
as a side effect of getting certificates.
"""

import json
import re
from datetime import datetime, timezone, timedelta

from core.finding import Finding
from core.knowledge import kb
from core.evidence import USER_AGENT

MODULE_WHOIS = "whois_lookup"
MODULE_CT = "cert_transparency"

try:
    import requests
    HAVE_REQUESTS = True
except ImportError:
    HAVE_REQUESTS = False

try:
    import whois as whois_lib
    HAVE_WHOIS = True
except ImportError:
    HAVE_WHOIS = False

def _mk(fid, module, target, observation, evidence=None, **kw):
    f = Finding(id=fid, module=module, target=target, title="",
               observation=observation, evidence=evidence or {}, **kw)
    return f.enrich(kb().get(fid))

def _as_date(value):
    if isinstance(value, list):
        value = value[0] if value else None
    if isinstance(value, datetime):
        return value.replace(tzinfo=value.tzinfo or timezone.utc)
    if isinstance(value, str):
        for fmt in ("%Y-%m-%dT%H:%M:%S", "%Y-%m-%d %H:%M:%S", "%Y-%m-%d"):
            try:
                return datetime.strptime(value[:19], fmt).replace(tzinfo=timezone.utc)
            except ValueError:
                continue
    return None

def run_whois(target, evidence_log=None, opts=None):
    findings = []
    if not HAVE_WHOIS:
        raise RuntimeError(
            "python-whois not installed. Run: pip install python-whois"
        )

    data = whois_lib.whois(target)
    if not data or not getattr(data, "domain_name", None):
        raise RuntimeError("No WHOIS data returned for {}".format(target))

    registrar = data.get("registrar") or "not disclosed"
    created = _as_date(data.get("creation_date"))
    expires = _as_date(data.get("expiration_date"))
    emails = data.get("emails") or []
    if isinstance(emails, str):
        emails = [emails]

    age = ""
    if created:
        days = (datetime.now(timezone.utc) - created).days
        age = " Domain age: {} days {:.1f} years".format(days, days / 365.25)

    findings.append(_mk(

```

```

        "WHO-001", MODULE_WHOIS, target,
        "Registrar: {}".format(
            registrar,
            created.date().isoformat() if created else "unknown",
            expires.date().isoformat() if expires else "unknown",
            age,
        ),
        {
            "registrar": registrar,
            "created": created.isoformat() if created else None,
            "expires": expires.isoformat() if expires else None,
            "emails": emails[:10],
            "name_servers": data.get("name_servers") or [],
        },
        passive=True,
    ))

    if expires and expires < datetime.now(timezone.utc) + timedelta(days=30):
        findings.append(_mk(
            "WHO-002", MODULE_WHOIS, target,
            "Registration expires {} - within 30 days.".format(
                expires.date().isoformat(),
                {"expires": expires.isoformat()}),
            passive=True,
        ))

    if evidence_log:
        evidence_log.record("whois", MODULE_WHOIS, target, {"registrar": registrar})

    return findings

def run_ct(target, rate=None, evidence_log=None, opts=None):
    """Query crt.sh certificate transparency logs for subdomains."""
    opts = opts or {}
    findings = []
    if not HAVE_REQUESTS:
        raise RuntimeError("requests not installed. Run: pip install requests")

    url = "https://crt.sh/?q=%25.{}&output=json".format(target)
    if rate:
        rate.wait("crt.sh")

    resp = requests.get(
        url, headers={"User-Agent": USER_AGENT},
        timeout=float(opts.get("timeout", 25)),
    )
    if resp.status_code == 429:
        retry = resp.headers.get("Retry-After", 10)
        if rate:
            rate.backoff("crt.sh", retry)
        raise RuntimeError("crt.sh rate limited (HTTP 429). Try again shortly.")
    if resp.status_code != 200:
        raise RuntimeError("crt.sh returned HTTP {}".format(resp.status_code))

    try:
        entries = resp.json()
    except json.JSONDecodeError:
        raise RuntimeError("crt.sh returned malformed JSON.")

    names = set()
    for entry in entries:
        for field in ("name_value", "common_name"):
            val = entry.get(field) or ""
            for line in str(val).split("\n"):
                line = line.strip().lower().rstrip("*.")
                if line.endswith(target) and re.match(r"^[a-z0-9._-]+$", line):
                    names.add(line)

    names.discard(target)
    ordered = sorted(names)

    if ordered:
        findings.append(_mk(
            "DNS-010", MODULE_CT, target,
            "{} unique hostname(s) recovered from certificate "
            "transparency logs.".format(len(ordered)),
            {"count": len(ordered), "hostnames": ordered[:200]},
            passive=True, confidence="confirmed",
        ))

    if evidence_log:
        evidence_log.record("ct_query", MODULE_CT, target, {"count": len(ordered)})

    return findings, ordered

```

modules/system/port_scan.py

ACTIVE - host discovery, TCP connect scan, banner grabbing.

```
"""
ACTIVE - host discovery, TCP connect scan, banner grabbing.

Every function here sends packets to the target and MUST pass the scope
check first. There is no bypass.

Design note for students: this is a TCP *connect* scan, meaning the full
three-way handshake completes before we close. That makes it reliable and
unprivileged, but also loud - the connection appears in the target's
application logs. A SYN scan avoids that by never completing the
handshake, but requires raw sockets and therefore root. The trade-off
between stealth and privilege is worth understanding directly.
"""

import socket
import ssl
import threading
from queue import Queue

from core.finding import Finding
from core.knowledge import kb

MODULE = "port_scan"

# Well-known ports with their teaching classification
SERVICES = {
    21: ("FTP", "cleartext"),
    22: ("SSH", "management"),
    23: ("Telnet", "cleartext"),
    25: ("SMTP", "mail"),
    53: ("DNS", "infrastructure"),
    69: ("TFTP", "cleartext"),
    80: ("HTTP", "web"),
    110: ("POP3", "cleartext"),
    111: ("rpcbind", "infrastructure"),
    135: ("MSRPC", "windows"),
    139: ("NetBIOS-SSN", "windows"),
    143: ("IMAP", "cleartext"),
    161: ("SNMP", "infrastructure"),
    389: ("LDAP", "directory"),
    443: ("HTTPS", "web"),
    445: ("SMB", "windows"),
    512: ("rexec", "cleartext"),
    513: ("rlogin", "cleartext"),
    514: ("rsh", "cleartext"),
    587: ("SMTP-submission", "mail"),
    631: ("IPP", "printing"),
    993: ("IMAPS", "mail"),
    995: ("POP3S", "mail"),
    1099: ("Java RMI", "application"),
    1433: ("MS-SQL", "database"),
    1521: ("Oracle DB", "database"),
    2049: ("NFS", "filesharing"),
    3000: ("HTTP-alt", "web"),
    3306: ("MySQL", "database"),
    3389: ("RDP", "management"),
    3632: ("distccd", "application"),
    4444: ("Metasploit default", "application"),
    5432: ("PostgreSQL", "database"),
    5900: ("VNC", "management"),
    5985: ("WinRM", "management"),
    6379: ("Redis", "database"),
    6667: ("IRC", "application"),
    8000: ("HTTP-alt", "web"),
    8009: ("AJP13", "application"),
    8080: ("HTTP-proxy", "web"),
    8081: ("HTTP-alt", "web"),
    8180: ("Tomcat", "web"),
    8443: ("HTTPS-alt", "web"),
    9200: ("Elasticsearch", "database"),
    11211: ("Memcached", "database"),
    27017: ("MongoDB", "database"),
}

TOP_PORTS = sorted(SERVICES.keys())

CLEARTEXT_PORTS = {p for p, (_, c) in SERVICES.items() if c == "cleartext"}
DATABASE_PORTS = {p for p, (_, c) in SERVICES.items() if c == "database"}
```

```

MANAGEMENT_PORTS = {p for p, (_, c) in SERVICES.items() if c == "management"}

WEB_PORTS = {p for p, (_, c) in SERVICES.items() if c == "web"}

def _mk(fid, target, observation, evidence=None, **kw):
    f = Finding(id=fid, module=MODULE, target=target, title="",
                observation=observation, evidence=evidence or {},
                passive=False, **kw)
    return f.enrich(kb().get(fid))

def parse_ports(spec):
    """Accept '80', '1-1024', '22,80,443', or 'top'."""
    if not spec or spec == "top":
        return list(TOP_PORTS)
    ports = set()
    for part in str(spec).split(","):
        part = part.strip()
        if "-" in part:
            lo, hi = part.split("-", 1)
            try:
                lo, hi = int(lo), int(hi)
            except ValueError:
                continue
            ports.update(range(max(1, lo), min(65535, hi) + 1))
        elif part.isdigit():
            ports.add(int(part))
    return sorted(ports)

def _connect(host, port, timeout):
    """Return (state, reason). state in open/closed/filtered."""
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(timeout)
    try:
        sock.connect((host, port))
        return "open", sock
    except socket.timeout:
        sock.close()
        return "filtered", None
    except ConnectionRefusedError:
        sock.close()
        return "closed", None
    except OSError as exc:
        sock.close()
        return "error:{}".format(exc.errno), None

def _grab_banner(sock, port, timeout=3):
    """Read whatever the service volunteers, nudging HTTP if silent."""
    banner = b""
    try:
        sock.settimeout(timeout)
        try:
            banner = sock.recv(512)
        except socket.timeout:
            banner = b""

        if not banner and port in WEB_PORTS:
            try:
                sock.sendall(b"HEAD / HTTP/1.0\r\n\r\n")
                banner = sock.recv(512)
            except (socket.timeout, OSError):
                banner = b""
    except OSError:
        pass
    return banner.decode("utf-8", errors="replace").strip()

def discover(host, scope, timeout=2.0, evidence_log=None):
    """Is the host up? Scope-gated."""
    scope.assert_in_scope(host)
    findings = []

    probe_ports = [80, 443, 22, 445, 3389, 21]
    for port in probe_ports:
        state, sock = _connect(host, port, timeout)

```

```

        if sock:
            sock.close()
        if state in ("open", "closed"):
            findings.append(_mk(
                "HOST-001", host,
                "Host responded on TCP/{0} ({1}), confirming it is live.".format(
                    port, state),
                {"probe_port": port, "state": state},
                confidence="confirmed",
            ))
        if evidence_log:
            evidence_log.record("host_up", MODULE, host, {"port": port})
        return findings, True

    return findings, False

def scan(host, scope, ports=None, timeout=1.5, threads=50,
        banners=True, evidence_log=None):
    """
    TCP connect scan. Scope check happens before any socket is opened.
    """
    scope.assert_in_scope(host)

    ports = ports or TOP_PORTS
    findings = []
    open_ports = []
    results_lock = threading.Lock()
    queue = Queue()
    for p in ports:
        queue.put(p)

    filtered_count = [0]
    closed_count = [0]

    def worker():
        while True:
            try:
                port = queue.get_nowait()
            except Exception:
                return
            try:
                state, sock = _connect(host, port, timeout)
                if state == "open":
                    banner = ""
                    if banners and sock:
                        banner = _grab_banner(sock, port)
                    if sock:
                        try:
                            sock.close()
                        except OSError:
                            pass
                    with results_lock:
                        open_ports.append((port, banner))
                elif state == "filtered":
                    with results_lock:
                        filtered_count[0] += 1
                elif state == "closed":
                    with results_lock:
                        closed_count[0] += 1
            finally:
                queue.task_done()

    pool = []
    for _ in range(min(threads, max(1, len(ports)))):
        t = threading.Thread(target=worker, daemon=True)
        t.start()
        pool.append(t)
    for t in pool:
        t.join()

    open_ports.sort()

    for port, banner in open_ports:
        name, category = SERVICES.get(port, ("unknown", "unknown"))

        findings.append(_mk(
            "PORT-001", host,

```

```

        "TCP/{ } open - likely { }. Full three-way handshake "
        "completed.".format(port, name),
        {"port": port, "service": name, "category": category},
        confidence="confirmed",
    ))

    if banner:
        clean = banner.replace("\r", " ").replace("\n", " ")[:200]
        findings.append(_mk(
            "PORT-002", host,
            "TCP/{ } banner: {}".format(port, clean),
            {"port": port, "banner": banner[:1000]},
            confidence="probable",
        ))

    if port in CLEARTXT_PORTS:
        findings.append(_mk(
            "PORT-003", host,
            "TCP/{ } ({} ) transmits data and credentials without "
            "encryption.".format(port, name),
            {"port": port, "service": name},
            confidence="confirmed",
        ))

    if port in DATABASE_PORTS:
        findings.append(_mk(
            "PORT-004", host,
            "TCP/{ } ({} ) is a database service reachable from this "
            "host.".format(port, name),
            {"port": port, "service": name},
            confidence="confirmed",
        ))

    if port in MANAGEMENT_PORTS:
        findings.append(_mk(
            "PORT-005", host,
            "TCP/{ } ({} ) provides remote administrative access.".format(
                port, name),
            {"port": port, "service": name},
            confidence="confirmed",
        ))

    if evidence_log:
        evidence_log.record("port_open", MODULE, host,
                           {"port": port, "banner": banner[:200]})

# Report the negative space too - it teaches port-state reasoning.
if filtered_count[0] or closed_count[0]:
    findings.append(_mk(
        "PORT-006", host,
        "{} port(s) closed (RST received), {} filtered (no response) "
        "out of {} probed.".format(
            closed_count[0], filtered_count[0], len(ports)),
        {"closed": closed_count[0], "filtered": filtered_count[0],
         "probed": len(ports)},
        confidence="confirmed",
    ))

return findings, [p for p, _ in open_ports]

```

modules/system/tls_audit.py

ACTIVE - certificate and TLS configuration audit.

```

"""
ACTIVE - TLS configuration and certificate audit.

Purely observational: we complete handshakes and read what the server
offers. No data is sent beyond the handshake itself.

The version-probing loop is a good classroom demonstration - it shows that
"which TLS versions does this server support" is not a single question but
one handshake attempt per version.
"""

import socket
import ssl
from datetime import datetime, timezone, timedelta

from core.finding import Finding
from core.knowledge import kb

MODULE = "tls_audit"

WEAK_CIPHER_MARKERS = ["RC4", "3DES", "DES", "NULL", "EXPORT", "MD5", "anon"]
NO_PFS_MARKERS = ["TLS_RSA_", "AES128-", "AES256-", "DES-CBC3-SHA"]

def _mk(fid, target, observation, evidence=None, **kw):
    f = Finding(id=fid, module=MODULE, target=target, title="",
                observation=observation, evidence=evidence or {},
                passive=False, **kw)
    return f.enrich(kb().get(fid))

def _parse_dn(dn_tuple):
    out = {}
    for rdn in dn_tuple or ():
        for key, val in rdn:
            out[key] = val
    return out

def _hostname_matches(hostname, cert):
    names = []
    for typ, val in cert.get("subjectAltName", ()) or ():
        if typ == "DNS":
            names.append(val.lower())
    subject = _parse_dn(cert.get("subject"))
    if subject.get("commonName"):
        names.append(subject["commonName"].lower())

    host = hostname.lower()
    for name in names:
        if name == host:
            return True, names
        if name.startswith("*."):
            suffix = name[1:]
            if host.endswith(suffix) and host.count(".") == name.count("."):
                return True, names
    return False, names

def audit(host, scope, port=443, timeout=8, evidence_log=None):
    """Full TLS audit. Scope-gated before any connection."""
    scope.assert_in_scope(host)
    findings = []

    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
    ctx.check_hostname = False
    ctx.verify_mode = ssl.CERT_NONE

    try:
        with socket.create_connection((host, port), timeout=timeout) as sock:
            with ctx.wrap_socket(sock, server_hostname=host) as tls:
                cert = tls.getpeercert()
                cipher = tls.cipher()
                version = tls.version()
    except (socket.timeout, ConnectionRefusedError, OSError, ssl.SSLError) as exc:
        raise RuntimeError("TLS handshake failed on {}:{} - {}".format(host, port, exc))

```

```

if not cert:
    raise RuntimeError("No certificate returned by {}:{}".format(host, port))

subject = _parse_dn(cert.get("subject"))
issuer = _parse_dn(cert.get("issuer"))
cn = subject.get("commonName", "unknown")
issuer_cn = issuer.get("commonName", issuer.get("organizationName", "unknown"))

san = [v for t, v in (cert.get("subjectAltName") or ()) if t == "DNS"]

not_after = None
if cert.get("notAfter"):
    try:
        not_after = datetime.strptime(
            cert["notAfter"], "%b %d %H:%M:%S %Y %Z"
        ).replace(tzinfo=timezone.utc)
    except ValueError:
        not_after = None

findings.append(_mk(
    "TLS-001", host,
    "Certificate for CN={} issued by {}. Valid until {}".format(
        cn, issuer_cn,
        not_after.date().isoformat() if not_after else "unknown"),
    {"subject": subject, "issuer": issuer,
     "not_after": cert.get("notAfter"), "san_count": len(san)},
    confidence="confirmed",
))

# --- expiry -----
now = datetime.now(timezone.utc)
if not_after:
    if not_after < now:
        findings.append(_mk(
            "TLS-002", host,
            "Certificate expired on {} ({} days ago)".format(
                not_after.date().isoformat(), (now - not_after).days),
            {"not_after": cert.get("notAfter")},
            confidence="confirmed",
        ))
    elif not_after < now + timedelta(days=30):
        findings.append(_mk(
            "TLS-003", host,
            "Certificate expires on {} ({} days remaining)".format(
                not_after.date().isoformat(), (not_after - now).days),
            {"not_after": cert.get("notAfter")},
            confidence="confirmed",
        ))

# --- hostname match -----
matched, names = _hostname_matches(host, cert)
if not matched and names:
    findings.append(_mk(
        "TLS-004", host,
        "Hostname '{}' not covered by certificate names: {}".format(
            host, ", ".join(names[:8])),
        {"requested": host, "cert_names": names},
        confidence="confirmed",
    ))

# --- self-signed -----
if subject and issuer and subject == issuer:
    findings.append(_mk(
        "TLS-005", host,
        "Certificate is self-signed (issuer equals subject: {})".format(cn),
        {"subject_cn": cn},
        confidence="confirmed",
    ))

# --- SAN disclosure -----
if len(san) > 1:
    findings.append(_mk(
        "TLS-009", host,
        "{} hostnames listed in the SAN extension: {}".format(
            len(san), ", ".join(san[:10]) + ("..." if len(san) > 10 else "")),
        {"san": san},
        confidence="confirmed",
    ))

```

```

# --- negotiated cipher -----
if cipher:
    cipher_name, cipher_proto, cipher_bits = cipher
    weak = [m for m in WEAK_CIPHER_MARKERS if m in cipher_name.upper()]
    no_pfs = any(cipher_name.startswith(m) for m in NO_PFS_MARKERS)
    if weak or no_pfs:
        reason = "weak algorithm ({})".format(", ".join(weak)) if weak \
        else "no forward secrecy"
        findings.append(_mk(
            "TLS-008", host,
            "Negotiated {} ({} bits) - {}".format(
                cipher_name, cipher_bits, reason),
            {"cipher": cipher_name, "bits": cipher_bits,
             "protocol": cipher_proto},
            confidence="confirmed",
        ))

# --- version probing -----
supported = _probe_versions(host, port, timeout)
obsolete = [v for v in supported if v in ("SSLv3", "TLSv1", "TLSv1.1")]
modern = [v for v in supported if v in ("TLSv1.2", "TLSv1.3")]

if obsolete:
    findings.append(_mk(
        "TLS-006", host,
        "Server accepts deprecated protocol version(s): {}".format(
            ", ".join(obsolete)),
        {"obsolete": obsolete, "all_supported": supported},
        confidence="confirmed",
    ))

if modern:
    findings.append(_mk(
        "TLS-007", host,
        "Server supports {} (negotiated {} on the default "
        "handshake)".format(", ".join(modern), version),
        {"modern": modern, "negotiated": version},
        confidence="confirmed",
    ))

if evidence_log:
    evidence_log.record("tls_audit", MODULE, host,
                        {"cn": cn, "versions": supported})

return findings

def _probe_versions(host, port, timeout):
    """One pinned handshake per version - the honest way to enumerate."""
    supported = []
    candidates = []

    for label, attr in [
        ("TLSv1", "TLSv1"), ("TLSv1.1", "TLSv1_1"),
        ("TLSv1.2", "TLSv1_2"), ("TLSv1.3", "TLSv1_3"),
    ]:
        ver = getattr(ssl.TLSVersion, attr, None)
        if ver is not None:
            candidates.append((label, ver))

    for label, ver in candidates:
        try:
            ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
            ctx.check_hostname = False
            ctx.verify_mode = ssl.CERT_NONE
            ctx.minimum_version = ver
            ctx.maximum_version = ver
            try:
                ctx.set_ciphers("ALL:@SECLEVEL=0")
            except ssl.SSLError:
                pass
            with socket.create_connection((host, port), timeout=timeout) as s:
                with ctx.wrap_socket(s, server_hostname=host):
                    supported.append(label)
        except Exception:
            continue
    return supported

```

modules/web/http_audit.py

ACTIVE - HTTP fingerprinting, security headers, cookies, forms.

```
"""
ACTIVE - HTTP fingerprinting, security header audit, form analysis.

All requests are ordinary GET/HEAD requests to documents the server is
already publishing. Nothing here submits data or attempts injection - the
tool stops at "here is the attack surface and its misconfigurations",
which is the correct boundary for a recon tool.
"""

import re
from urllib.parse import urljoin, urlparse

from core.finding import Finding
from core.knowledge import kb
from core.evidence import USER_AGENT

MODULE = "http_audit"

try:
    import requests
    HAVE_REQUESTS = True
except ImportError:
    HAVE_REQUESTS = False

try:
    from bs4 import BeautifulSoup
    HAVE_BS4 = True
except ImportError:
    HAVE_BS4 = False

DISCLOSURE_HEADERS = [
    "x-powered-by", "x-aspnet-version", "x-aspnetmvc-version",
    "x-generator", "x-drupal-cache", "x-runtime", "x-version",
]

def _mk(fid, target, observation, evidence=None, **kw):
    f = Finding(id=fid, module=MODULE, target=target, title="",
                observation=observation, evidence=evidence or {},
                passive=False, **kw)
    return f.enrich(kb().get(fid))

def _session(timeout):
    s = requests.Session()
    s.headers.update({"User-Agent": USER_AGENT})
    s.verify = False
    return s

def audit(url, scope, timeout=10, rate=None, evidence_log=None):
    """Fetch the URL and audit headers, cookies and forms."""
    if not HAVE_REQUESTS:
        raise RuntimeError("requests not installed. Run: pip install requests")

    parsed = urlparse(url if "://" in url else "http://" + url)
    host = parsed.hostname
    scope.assert_in_scope(host)

    import urllib3
    urllib3.disable_warnings()

    findings = []
    sess = _session(timeout)
    if rate:
        rate.wait(host)

    resp = sess.get(url, timeout=timeout, allow_redirects=True)
    headers = {k.lower(): v for k, v in resp.headers.items()}

    findings.append(_mk(
        "HTTP-001", url,
        "HTTP {} received. Final URL after redirects: {}".format(
            resp.status_code, resp.url),
        {"status": resp.status_code, "final_url": resp.url,
         "length": len(resp.content),
         "redirect_chain": [r.url for r in resp.history]},
    ))
```

```

        confidence="confirmed",
    ))

# --- server / stack disclosure -----
server = headers.get("server")
if server:
    has_version = bool(re.search(r"\d+\.\d+", server))
    findings.append(_mk(
        "HTTP-002", url,
        "Server header: {}".format(
            server,
            " (includes a version number)" if has_version else
            " (version suppressed)"
        ),
        {"server": server, "version_disclosed": has_version},
        confidence="probable",
    ))

stack = {h: headers[h] for h in DISCLOSURE_HEADERS if h in headers}
if stack:
    findings.append(_mk(
        "HTTP-003", url,
        "Technology headers present: {}".format(
            "; ".join("{}: {}".format(k, v) for k, v in stack.items())
        ),
        {"headers": stack},
        confidence="probable",
    ))

# --- HTTP -> HTTPS redirect -----
if parsed.scheme == "http":
    if not resp.url.lower().startswith("https://"):
        findings.append(_mk(
            "WEB-001", url,
            "Plain HTTP request returned content (HTTP {}) without "
            "redirecting to HTTPS.".format(resp.status_code),
            {"final_url": resp.url, "status": resp.status_code},
            confidence="confirmed",
        ))

# --- security headers -----
csp = headers.get("content-security-policy", "")

if parsed.scheme == "https" or resp.url.startswith("https://"):
    if "strict-transport-security" not in headers:
        findings.append(_mk(
            "HDR-001", url,
            "No Strict-Transport-Security header in the response.",
            {}, confidence="confirmed",
        ))
    else:
        findings.append(_mk(
            "HDR-006", url,
            "Strict-Transport-Security present: {}".format(
                headers["strict-transport-security"]
            ),
            {"value": headers["strict-transport-security"]},
            confidence="confirmed",
        ))

if not csp:
    findings.append(_mk(
        "HDR-002", url,
        "No Content-Security-Policy header in the response.",
        {}, confidence="confirmed",
    ))
else:
    weak = []
    if "unsafe-inline" in csp:
        weak.append("unsafe-inline")
    if "unsafe-eval" in csp:
        weak.append("unsafe-eval")
    f = _mk(
        "HDR-006", url,
        "Content-Security-Policy present{}".format(
            " but weakened by {}".format(", ".join(weak)) if weak else ""
        ),
        {"csp": csp[:500], "weak_directives": weak},
        confidence="confirmed",
    )
    if weak:
        f.severity = "low"

```

```

        f.title = "Content-Security-Policy present but permissive"
        findings.append(f)

    if "x-frame-options" not in headers and "frame-ancestors" not in csp:
        findings.append(_mk(
            "HDR-003", url,
            "Neither X-Frame-Options nor a CSP frame-ancestors directive "
            "is present.",
            {}, confidence="confirmed",
        ))

    if headers.get("x-content-type-options", "").lower() != "nosniff":
        findings.append(_mk(
            "HDR-004", url,
            "X-Content-Type-Options is missing or not set to 'nosniff'.",
            {"value": headers.get("x-content-type-options")},
            confidence="confirmed",
        ))

    # --- cookies -----
    for cookie in resp.cookies:
        missing = []
        if not cookie.secure:
            missing.append("Secure")
        rest = {k.lower(): v for k, v in (cookie._rest or {}).items()}
        if "httponly" not in rest:
            missing.append("HttpOnly")
        if "samesite" not in rest:
            missing.append("SameSite")
        if missing:
            findings.append(_mk(
                "HDR-005", url,
                "Cookie '{}' set without: {}".format(
                    cookie.name, ", ".join(missing)),
                {"cookie": cookie.name, "missing_flags": missing},
                confidence="confirmed",
            ))

    # --- robots.txt -----
    try:
        robots_url = urljoin(resp.url, "/robots.txt")
        if rate:
            rate.wait(host)
        r = sess.get(robots_url, timeout=timeout)
        if r.status_code == 200 and "user-agent" in r.text.lower():
            disallowed = re.findall(r"(?im)^\s*Disallow:\s*(\S+)", r.text)
            if disallowed:
                findings.append(_mk(
                    "WEB-004", robots_url,
                    "{} Disallow entries published: {}".format(
                        len(disallowed), ", ".join(disallowed[:12])),
                    {"disallowed": disallowed[:60]},
                    confidence="confirmed",
                ))
    except requests.RequestException:
        pass

    # --- forms -----
    if HAVE_BS4 and resp.text:
        findings.extend(_analyse_forms(resp.text, resp.url))

    if evidence_log:
        evidence_log.record("http_audit", MODULE, url,
                           {"status": resp.status_code, "server": server})

    return findings

def _analyse_forms(body, base_url):
    findings = []
    try:
        soup = BeautifulSoup(body, "html.parser")
    except Exception:
        return findings

    for form in soup.find_all("form"):
        method = (form.get("method") or "GET").upper()
        action = form.get("action") or ""

```

```
action_url = urljoin(base_url, action)

inputs = []
has_password = False
has_csrf = False
for inp in form.find_all(["input", "select", "textarea"]):
    itype = (inp.get("type") or "text").lower()
    iname = inp.get("name") or ""
    inputs.append({"name": iname, "type": itype})
    if itype == "password":
        has_password = True
    if re.search(r"csrf|token|authenticity|nonce", iname, re.I):
        has_csrf = True

note = ""
if method == "POST" and not has_csrf:
    note = " No CSRF token field detected on this state-changing form."

findings.append(_mk(
    "WEB-005", action_url,
    "{} form with {} input(s) -> {}".format(
        method, len(inputs), action_url, note),
    {"method": method, "action": action_url, "inputs": inputs,
     "csrf_field_present": has_csrf},
    confidence="confirmed",
))

if has_password and action_url.lower().startswith("http://"):
    findings.append(_mk(
        "WEB-006", action_url,
        "Form containing a password field submits to a cleartext "
        "HTTP URL: {}".format(action_url),
        {"action": action_url},
        confidence="confirmed",
    ))

return findings
```

modules/web/content_discovery.py

ACTIVE - wordlist path discovery with baseline comparison.

```
"""
ACTIVE - content discovery (forced browsing).

Teaching emphasis: the baseline. A naive scanner reports every 200 status
as a hit, which produces pages of noise against any server with a custom
404 handler. Before probing anything, this module requests a path that
certainly does not exist and records what the server does. Every
subsequent response is compared against that baseline.

Rate limiting is mandatory here, not optional - unthrottled content
discovery is indistinguishable from a denial-of-service attempt.
"""

import hashlib
import re
import time
from urllib.parse import urljoin, urlparse

from core.finding import Finding
from core.knowledge import kb
from core.evidence import USER_AGENT

MODULE = "content_discovery"

try:
    import requests
    HAVE_REQUESTS = True
except ImportError:
    HAVE_REQUESTS = False

DEFAULT_PATHS = [
    "admin/", "administrator/", "login", "login.php", "wp-admin/",
    "wp-login.php", "phpmyadmin/", "manager/html", "cgi-bin/",
    "console", "dashboard", "panel", "portal",
    ".git/HEAD", ".git/config", ".svn/entries", ".hg/",
    ".env", ".env.local", ".env.backup",
    "config.php", "config.php.bak", "configuration.php",
    "web.config", "settings.py", "application.yml",
    "backup/", "backup.zip", "backup.tar.gz", "db.sql", "dump.sql",
    "database.sql", "site.zip", "www.zip", "old/", "bak/", "tmp/",
    "test/", "dev/", "staging/", "beta/",
    "server-status", "server-info", "status", "health", "metrics",
    "api/", "api/v1/", "api/docs", "swagger.json", "swagger-ui.html",
    "openapi.json", "graphql",
    "robots.txt", "sitemap.xml", "crossdomain.xml", "security.txt",
    ".well-known/security.txt", "readme.html", "README.md",
    "info.php", "phpinfo.php", "test.php",
    "uploads/", "files/", "images/", "assets/", "static/",
    "logs/", "log/", "error.log", "access.log", "debug.log",
]

INDEX_MARKERS = [
    "index of /", "<title>index of", "directory listing for",
    "[to parent directory]", "parent directory</a>",
]

def _mk(fid, target, observation, evidence=None, **kw):
    f = Finding(id=fid, module=MODULE, target=target, title="",
                observation=observation, evidence=evidence or {},
                passive=False, **kw)
    return f.enrich(kb().get(fid))

def _fingerprint(resp):
    """Status + rounded length + body hash: enough to spot soft-404s."""
    body = resp.content or b""
    return {
        "status": resp.status_code,
        "length": len(body),
        "hash": hashlib.sha256(body).hexdigest()[16:],
    }

def _load_wordlist(path):
    if not path:
        return list(DEFAULT_PATHS)
```

```

try:
    with open(path, "r", encoding="utf-8", errors="ignore") as fh:
        words = [ln.strip() for ln in fh if ln.strip() and not ln.startswith("#")]
        return words or list(DEFAULT_PATHS)
except OSError:
    return list(DEFAULT_PATHS)

def discover(base_url, scope, wordlist=None, delay=0.35, timeout=8,
            max_paths=400, evidence_log=None, progress=None):
    """
    Wordlist-driven path discovery with baseline comparison.
    Scope-gated. Rate-limited by construction.
    """
    if not HAVE_REQUESTS:
        raise RuntimeError("requests not installed. Run: pip install requests")

    parsed = urlparse(base_url if "://" in base_url else "http://" + base_url)
    host = parsed.hostname
    scope.assert_in_scope(host)

    import urllib3
    urllib3.disable_warnings()

    if not base_url.endswith("/"):
        base_url += "/"

    sess = requests.Session()
    sess.headers.update({"User-Agent": USER_AGENT})
    sess.verify = False

    findings = []

    # --- establish the baseline -----
    baselines = []
    for probe in ("edrecon-baseline-a9f3c1x/", "edrecon-baseline-7k2m8q.html"):
        try:
            r = sess.get(urljoin(base_url, probe), timeout=timeout,
                          allow_redirects=False)
            baselines.append(_fingerprint(r))
        except requests.RequestException:
            pass
        time.sleep(delay)

    baseline_statuses = {b["status"] for b in baselines}
    baseline_hashes = {b["hash"] for b in baselines}
    baseline_lengths = {b["length"] for b in baselines}

    soft_404 = 200 in baseline_statuses
    if soft_404:
        findings.append(_mk(
            "HTTP-001", base_url,
            "Server returns HTTP 200 for non-existent paths (soft 404). ",
            "Results below are filtered by body hash and length, not ",
            "status code alone.",
            {"baseline": baselines},
            confidence="confirmed",
        ))

    words = _load_wordlist(wordlist)[:max_paths]
    total = len(words)

    for idx, word in enumerate(words, 1):
        if progress and idx % 25 == 0:
            progress(idx, total)

        url = urljoin(base_url, word.lstrip("/"))
        try:
            r = sess.get(url, timeout=timeout, allow_redirects=False)
        except requests.RequestException:
            time.sleep(delay)
            continue

        time.sleep(delay)

        if r.status_code == 429:
            retry = r.headers.get("Retry-After", 5)
            try:

```

```

        time.sleep(min(float(retry), 30))
    except (TypeError, ValueError):
        time.sleep(5)
    continue

fp = _fingerprint(r)

# Filter against baseline
if fp["hash"] in baseline_hashes:
    continue
if r.status_code in (404, 400):
    continue
if soft_404 and fp["length"] in baseline_lengths:
    continue
if r.status_code not in (200, 201, 204, 301, 302, 307, 401, 403, 500):
    continue

body_l = (r.text or "")[:4000].lower()

# Directory listing?
if r.status_code == 200 and any(m in body_l for m in INDEX_MARKERS):
    findings.append(_mk(
        "WEB-003", url,
        "Directory listing enabled at {} (HTTP 200)".format(url),
        {"url": url, "length": fp["length"]},
        confidence="confirmed",
    ))
    continue

# Version control exposure?
if re.search(r"/\.(git|svn|hg)/", url) and r.status_code == 200:
    findings.append(_mk(
        "WEB-007", url,
        "Version control artefact reachable: {} (HTTP 200, "
        "{} bytes)".format(url, fp["length"]),
        {"url": url, "length": fp["length"],
         "preview": (r.text or "")[:200]},
        confidence="confirmed",
    ))
    continue

# Generic hit
interpretation = {
    200: "exists and is readable",
    201: "exists",
    204: "exists, empty response",
    301: "redirects to {}".format(r.headers.get("Location", "?")),
    302: "redirects to {}".format(r.headers.get("Location", "?")),
    307: "redirects to {}".format(r.headers.get("Location", "?")),
    401: "exists but requires authentication",
    403: "exists but access is forbidden - the path is confirmed",
    500: "exists and triggered a server error",
}.get(r.status_code, "responded")

f = _mk(
    "WEB-002", url,
    "{} -> HTTP {} ({}), {} bytes".format(
        url, r.status_code, interpretation, fp["length"]),
    {"url": url, "status": r.status_code, "length": fp["length"],
     "content_type": r.headers.get("Content-Type", "")},
    confidence="confirmed",
)
if r.status_code in (401, 403):
    f.severity = "low"
findings.append(f)

if evidence_log:
    evidence_log.record("path_found", MODULE, url,
                       {"status": r.status_code, "length": fp["length"]})

return findings

```

scope.example.yaml

Scope file template. Copy and edit for your lab.

```
# =====
# EDRecon Authorisation Scope File
# -----
# This file IS the authorisation record. EDRecon refuses to run any active
# module against a target not listed here, and there is no override flag.
#
# In a real engagement this mirrors the signed rules of engagement.
# In the classroom it mirrors the lab sheet. Treating it as mandatory is
# the habit that transfers to professional practice.
#
# Copy this file, edit it, and pass it with --scope.
# =====

engagement:
  # All four fields are required. EDRecon will not start without them.
  name: "Ethical Hacking Lab - Unit 2 Reconnaissance"
  authorised_by: "Dr. Keshav Sinha, School of Computer Science, UPES"
  contact: "keshav.sinha@ddn.upes.ac.in"

  # YYYY-MM-DD. An expired scope is refused, exactly like real authorisation.
  # Set this to the last day of your lab block.
  expires: "2026-12-31"

  notes: >
    Isolated lab environment. Metasploitable 3 target VM on a host-only
    network. No internet-facing systems are in scope. Students must not
    modify this file to add targets outside the lab network.

in_scope:
  # Individual hosts, by IP or hostname.
  hosts:
    - "192.168.56.101"      # Metasploitable 3 (Ubuntu)
    - "192.168.56.102"      # Metasploitable 3 (Windows)

  # Whole networks in CIDR notation. Use this for the lab subnet.
  networks:
    - "192.168.56.0/24"      # VirtualBox host-only network

  # Domains. Subdomains are automatically included.
  domains: []
    # - "lab.example.local"

# Anything listed here is refused even if it also matches in_scope above.
# Exclusions always win. Use this to carve out the gateway, the host
# machine, or any device that must not be touched.
out_of_scope:
  - "192.168.56.1"          # VirtualBox host adapter - do not scan
  - "192.168.56.100"        # Kali attacker VM - that is you

# =====
# TEMPLATE FOR A SCOPE FILE OF YOUR OWN
# -----
# engagement:
#   name: "<what this assessment is>"
#   authorised_by: "<person who signed off, with role>"
#   contact: "<their email>"
#   expires: "<YYYY-MM-DD, the day authorisation lapses>"
#
# in_scope:
#   hosts: ["<ip or hostname>"]
#   networks: ["<cidr>"]
#   domains: ["<domain>"]
#
# out_of_scope:
#   - "<anything explicitly forbidden>"
# =====
```

requirements.txt

Python dependencies.

```
# EDRecon requirements
# Install with: pip install -r requirements.txt

# --- required -----
PyYAML>=6.0          # scope file + knowledge base parsing
requests>=2.31.0     # HTTP modules, certificate transparency
dnspython>=2.4.0     # DNS reconnaissance

# --- recommended -----
beautifulsoup4>=4.12.0 # form analysis in the web phase
python-whois>=0.9.0    # WHOIS registration lookups
urllib3>=2.0.0        # TLS warning suppression for lab self-signed certs
```